Tutorial Lab 12.5 (Formatively Assessed):

Cascode Stages, Mirrors, and Symbol View Generation

Preliminary Matter: Setting up the Cadence Environment

Please read through sections 1 through 3 of the tutorial lab 12.1 instruction booklet/handout before entering these commands into your terminal.

For this course, you will use the Cadence Virtuoso Design Environment (referred to simply as Cadence). Cadence is a Linux based software suite for designing full custom integrated circuits. In order to use Cadence you will first need to set-up the Cadence environment.

1. Open a terminal window and ensure that you are in your home directory using the following command:

2.

```
cd ~
```

Note: 'cd' stands for change directory. The solitary tilde symbol (~) is a shortcut/alias for your home directory (/home/username).

3. Under your home directory, use the following commands to load the Cadence environment:

```
module use /eda/cadence/modules

module load course/EENGM6030
```

Note: The two 'module' commands load the required shell/Cadence environmental variables for this course (paths, names, aliases…) In particular, this includes a reference to the process design kit (PDK) used in this course, the GPDK180. You will need to do this every time you open a terminal, since the sourced environment only applies to the current terminal (until it's closed).

a. ***Note: This step only needs to be completed ONCE - If you already ran this script in lab 12.1, you don't need to run it again here.*** The first time you use Cadence, you will need to run the following command (script):

```
EENGM6030_setup
```

This will create a folder called AICD (short for Analogue Integrated Circuit Design) under your home directory. The AICD folder contains the files and directories needed to run Cadence. Running this setup script more than once will result in a warning message that the AICD folder already exists. It will then exit without making changes.

4. The AICD folder will be used for all your Cadence related activities for this course. **You should always make sure you are in your AICD folder before you launch Cadence.** To run cadence, use the following commands:

```
cd ~/AICD

virtuoso &
```

1. Learning Objectives of this Lab:

From the previous lab, you should:

- Understand the importance of feedback for control and compensation in electronic circuits and the potential for instability within such circuits. Understand how the RC products (delays) in the forward and feedback paths of a circuit give rise to poles and zeros and how these affect the frequency response of the feedback circuit.

- Know how to use the SPICE/Spectre method for stability analysis (**stb**) in feedback circuits. Understand the role of Loop Gain Magnitude and Loop Gain Phase in stability analysis. Understand the Phase Margin and Gain Margin stability metrics and how to obtain them through simulation.

- Use feedback to provide voltage regulation.

In this lab, you will:

- Understand the concept of channel length modulation.

- Understand the effect of channel length on MOSFET output impedance.

- Understand the concept of cascoding to increase MOSFET output impedance.

- Simulate a telescopic cascode amplifier circuit and compare its performance with that of a non-cascode equivalent. Investigate cascoding in a FET current mirror.

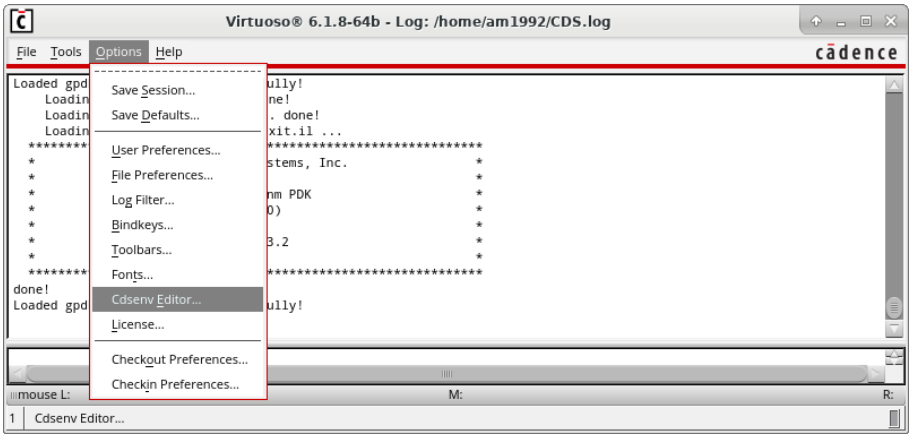
- Construct, instantiate and use in a simulation, a symbol view for a schematic circuit.

2. Cadence Documentation, Help, and Licensing

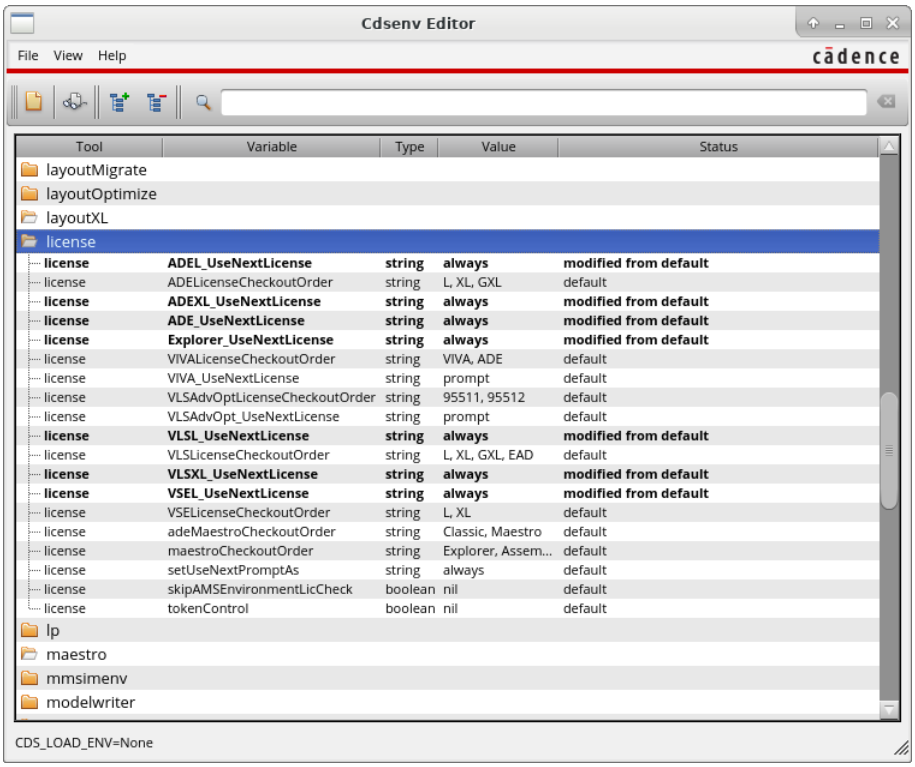
Local and online help is available from the help menus (**Menu: Help → User Guide** or by pressing **F1**), or by typing 'cdnshelp &' at the command prompt (assuming you have setup your terminal environment).

PLEASE NOTE: WHEN LAUNCHING A TOOL IN THE CADENCE DESIGN ENVIRONMENT YOU MAY SOMETIMES GET A WARNING ABOUT A LICENSE BEING UNAVAILABLE FOR THE TOOL, AND REQUESTING PERMISSION TO OBTAIN A HIGHER-TIERED LICENSE. YOU SHOULD SELECT 'ALWAYS' TO THESE MESSAGES.

If you accidently deny a license request, or you cannot access a license, you will need to modify the Cadence environment licence/permissions from the CIW. From the CIW menu go to **Menu: Options → Cdsenv Editor**. Once in the environment editor, go to the license folder, and make sure all licenses (in bold) are set to '**always'** – if any entries are listed as '**never**' you will need to manually change/retype them to '**always**'.



< Figure X.1. The Cadence Environment Menu Option from the CIW Window. >



< Figure X.2. The Cadence Environment Editor. Note, all licenses should be set to 'always'.

3. Getting Started with Cadence:

For a refresher/introduction on how to start a Cadence session please read through sections 1 through 3 of the tutorial lab 12.1 instruction booklet/handout.

3.1 Dealing with Lock files

PLEASE NOTE: IF A SCHEMATIC ISN'T PROPERLY CLOSED DOWN (FOR EXAMPLE, DUE TO A DROPPED REMOTE CONNECTION), THE SCHEMATIC MIGHT BECOME EDIT LOCKED. IF THIS HAPPENS YOU WILL NOT BE ABLE TO OPEN THE FILE FOR EDITING. YOU WILL ONLY BE ABLE TO OPEN THE FILE FOR READING UNTIL YOU REMOVE ANY/ALL LOCK FILES.

There are two ways you can unlock files:

(1) The first is to manually delete any edit lock files. To do this, go to your '~/AICD/Designs' folder and find the folder representing the schematic e.g., for lab1 this would be '~/AICD/Designs/lab121_vdivider/schematic' if this is what you called your schematic. Delete all the .cdslck lock files in this folder and you should be able to open your schematic for editing.

(2)

(3) You can also automatically release edit locks with the command:

```
clsAdminTool -are .
```

To use this command, you have to be in your AICD folder (the full-stop is shorthand for the current directory path). If you are not in your AICD folder, you have to specify the path (relative or absolute) to your AICD directory.

Since this is a Cadence command, you have to ensure that you have set up the terminal environment for Cadence (in particular, the modules must be loaded as in step 2, 'Preliminary Matter'). It is best to use the terminal window that you used to start Cadence, since the environment was already set up. Note that you will not be able enter any commands into your terminal if you did not start cadence with the '&' character. By example, to release any lock files after opening a new terminal, the full command set would be,

```
cd~
module use /eda/cadence/modules
module load course/EENGM6030_2021
clsAdminTool -are AICD
```

For more information on this command, use

```
clsAdminTool -help
```

4. Circuit Simulation Exercises: Part 1

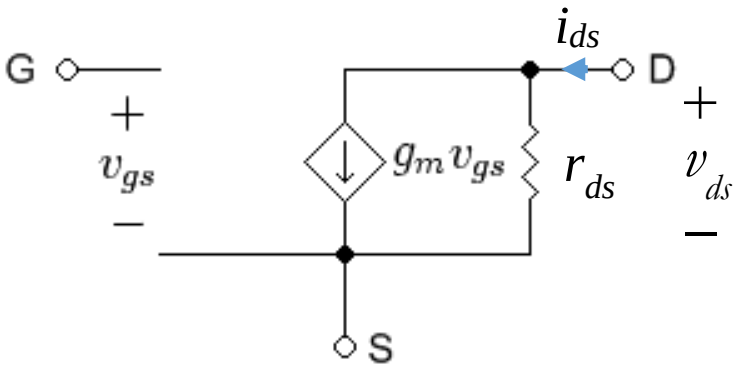
With guidance, you will:

- Understand the concept of channel length modulation. Simulate the dependency of the MOSFET output impedance on channel length.

- Understand the concept of cascoding. Simulate a telescopic cascode testbench circuit and compare its performance with that of a non-cascode equivalent.

The ideal MOSFET – modelled as a Voltage-Controlled Current-Source (VCCS) – has an infinite incremental output impedance. In this case, the drain (output) current I_{DS} is independent of the drain (output) voltage V_{DS}. As we have seen in the previous labs, the non-ideal MOSFET deviates from the VCCS model by a dependency of the output current on the output voltage. This non-ideal behaviour is modelled in the small-signal domain by a parasitic resistance r_{ds} (aka r_o) between the output terminals of the VCCS, where $r_{ds} = dV_{DS}/dI_{DS}$. At the heart of this behaviour is the concept of channel length modulation, where the effective length of the channel varies with drain-to-source voltage due to the electrical field moving the channel pinch-off point away from the drain. (We will discuss channel length modulation later in this tutorial, but for a summary of the phenomena refer to section 1.2.5 of the background material and section 4.2.3 in reference [3]).

In the previous labs, we have characterised – through simulation – the V_{DS}-I_{DS} large-signal terminal behaviour of an NMOS device from the GPDK180 library, and from this, plotted r_{ds} as a function of the DC operating point. Consider, the simplest small-signal model of the MOSFET



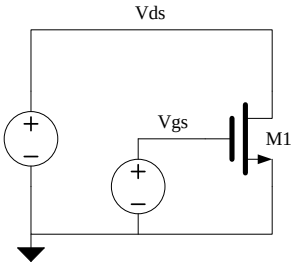
< Figure 117. Voltage controlled current source with (non-ideal) output impedance r_{ds}. >

Ultimately, the presence of r_{ds} will lower the voltage gain of the MOSFET amplifier for a given transconductance as it appears in parallel with the infinite output impedance of the ideal MOSFET (and also load device). In this lab, we will focus on mitigating the dependency of I_{DS} on V_{DS} in order to increase the output impedance of the MOSFET – and ultimately the gain of the MOSFET voltage amplifier. We will introduce the concept of channel length modulation and observe – through simulation – how the choice of channel length effects the output impedance of the MOSFET. We will also introduce the circuit technique of amplifier cascoding

as a way of increasing the MOSFET output impedance, and how this technique avoids the adverse consequences of selecting a longer channel length.

4.1 NMOS Terminal Characteristics

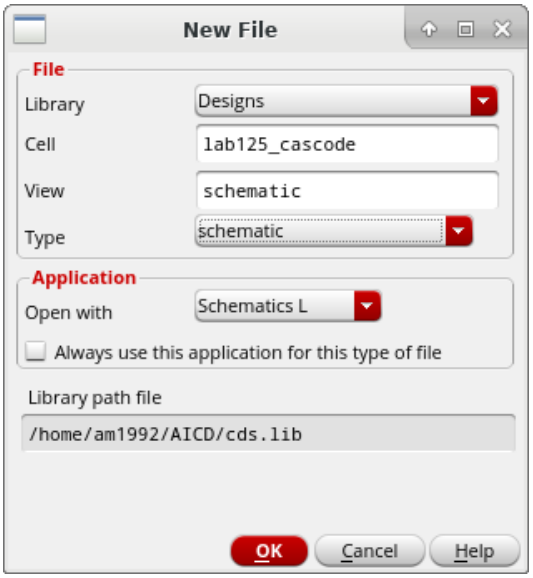
(4.1.1) Create the schematic for the NMOS testbench circuit in Figure 2. A summary of the circuit's operation is given in section 4.1 of the background material. Note: You will have created this circuit in labs 12.2 and 12.3. You can re-use your circuit, copy it, or build it from scratch. We recommend that you create a new schematic since this is a new lab (you should be able to do this quickly).



< Figure 118. Circuit for Large Signal Transconductance Terminal Characteristics Evaluation.
Reproduced from Figure 14, section 4.1 of the background material. >

(4.1.2) From the Library Manager window, select the 'Designs' library and choose **Menu: Files à New à Cell View** to invoke the New File form (Figure 3).

In the 'Cell' text field, choose a meaningful cell name (e.g. lab125_cascode). In the 'View' text field, type 'schematic' as the view name and from the 'Type' drop down menu choose 'schematic' as the view type. In the 'Application' section of the dialog box, choose 'Schematics L' from the 'Open with' drop down menu. OK the form.



< Figure 119. The New File Form. >

(4.1.3) You should be presented with a blank **Virtuoso Schematic Editor L** schematic entry window for the new cellview. This cellview will not be saved in the Library Manager until you save it in the Schematic Editor. From the schematic editor window, use either **Menu: File à Save** or **Menu: File à Check and Save**.

(4.1.4) In your schematic editor window, construct the circuit given in Figure 2. Instantiate and place two DC voltage sources (**vdc**) from the analogLib library, a ground connection (**gnd**) from the analogLib library, and one high-voltage NMOS FET (**nmoshv**) from the GPDK180 library.

(4.1.5) Parameterise your design. For the DC supply voltage, set the voltage attribute to the string vds (**DC Voltage** = *vds*) and for the gate-to-source voltage set the voltage attribute to the string vgs (**DC Voltage** = *vgs*), and dimension the NMOS device 10um/1um (**width** = *10u*, **length** = 1u). Leave all other attributes unchanged.

(4.1.6) With the components placed, you can now add wires to connect them together. You can use **Menu: Create → Wire (narrow)**, the '**Create Narrow Wire**' icon-button on the toolbar, or the keyboard shortcut '**W**' to start the wire command (see Figure 5).

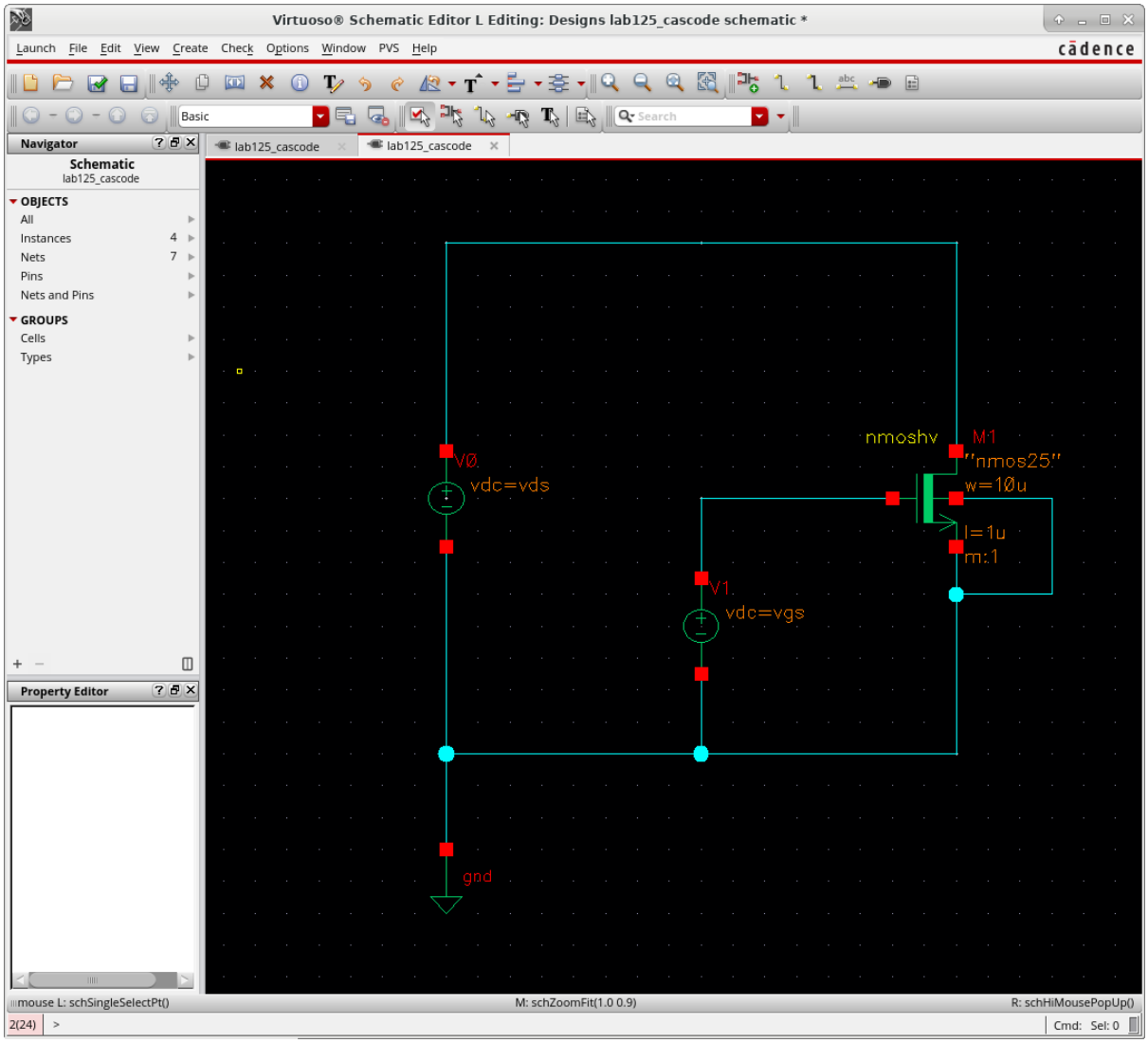
(4.1.7) Check and save your schematic using the **Check and Save** command. Remember to use the output area of the CIW to diagnose any problems with your schematic and/or simulations if there are any warning or error messages when checking, saving, or simulating your work.
*Recall, to add a new part you can use **Menu: Create → Instance...**, the '**Create Instance**' icon-button on the toolbar at the top of the schematic window, or the keyboard shortcut '**I**'. You can also right-click anywhere in the empty space of the design canvas to invoke a context-menu and select the **Add Instance…** command from there. Refer to lab 12-1 for a refresher on how to add components to the schematic (sec. 4.2 Adding Components to the Schematic).*

*You can set the component properties at instantiation from the Add Instance form before placing them on the design canvas. Alternatively, you can modify the component properties after it has been placed on the design canvas, by first selecting the item and then by right-clicking on the item and selecting 'Properties…' You can also use **Menu: Edit → Properties → Objects**, the '**Edit Properties**' icon-button from the toolbar, or the keyboard shortcut '**Q**'. You can then modify the required attribute(s) and Apply/OK the change.*

*You can drag, move or stretch the components and wires after they have been placed on the design canvas. Left-click-and-hold the mouse button and drag the component or wire, or select the component and use the **stretch** or **move** commands. You can also rotate and flip the components before placing them using the **rotate**, **sideways**, and **upside-down** buttons on the add instance form. Or you can rotate/flip the components after placing them by selecting the component and using the **rotate** or **flip** commands, or by using **F3** with the **move/stretch** commands.*



< Figure 120. The nmoshv (GPDK180) and vdc (analogLib) Add Instance Forms. >



< Figure 121. The wired NMOS testbench circuit with Design Variables for the DC supply voltage (vdc=vds) and the DC gate-to-source voltage (vdc=vgs). nmoshv=10um/1um. >

(4.1.8) RECALL: FOR AN NMOS DEVICE, YOU MUST ALWAYS CONNECT THE BODY/BULK TERMINAL TO THE MOST NEGATIVE NET (WIRE) OF THE CIRCUIT. IN FIGURE 5, THIS IS THE GROUND TERMINAL. FOR A PMOS DEVICE (NOT SHOWN) THE BODY/BULK TERMINAL MUST ALWAYS BE CONNECTED TO THE MOST POSITIVE NET (WIRE) OF THE CIRCUIT, USUALLY THE SUPPLY VDD. THIS IS NECESSARY TO ENSURE THAT THE CHANNEL WILL BE PROPERLY ESTABLISHED.

4.2 Simulating the NMOS FET DC operating Point

(4.2.1) You should now have the completed schematic of the NMOS testbench circuit as shown in Figure 5. It should check and save without any errors or warnings.

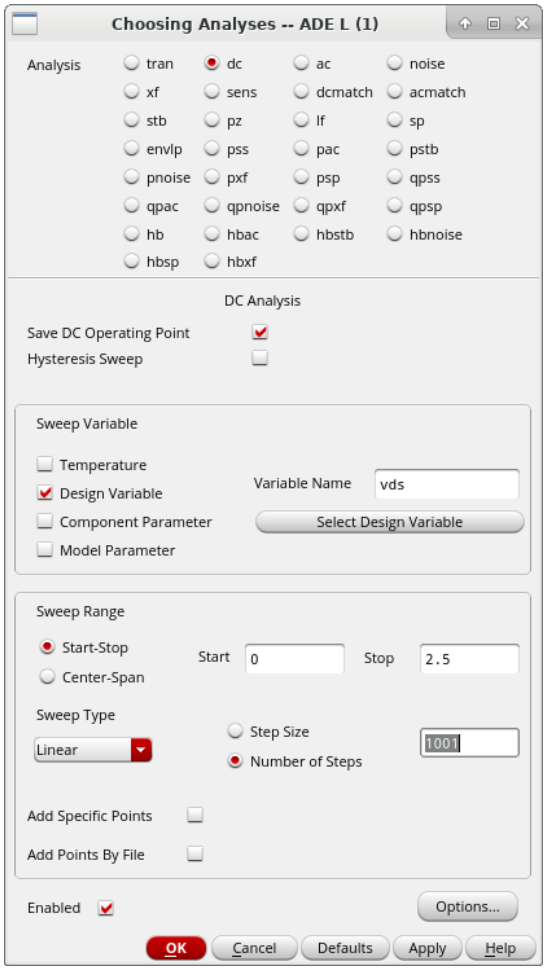
(4.2.2) We will simulate the NMOS testbench using a DC Sweep Analysis. From the Schematic Editor window, launch the Analogue Design Environment (ADE) – **Menu: Launch à ADE L**.

(4.2.3) Copy the design variables from the schematic into the ADE. For this circuit, these are the DC voltages from the two **vdc** components - *vds* and *vgs*. From the ADE window, choose **Menu: Variables à Copy From Cellview**. You should now see the *vds* and *vgs* variables in the 'Design Variables' list of the ADE. Initialise *vds = 2.5V* and *vgs = 0.7V*. These values are used only to calculate the DC operating point of the amplifier circuit.

(4.2.4) We want to plot the I_{DS} vs. V_{DS} family of curves for increasing values of V_{GS} (that is I_{DS} vs. V_{DS} vs. V_{GS}). This requires two sweeps – for V_{DS} from 0V to 2.5V and for V_{GS} from 0V to 2.5V. The DC analysis form only allows you to sweep one variable at a time, so we will also need to use the Parametric Analysis tool as we did in lab 12.2.

(4.2.5) From the ADE window, go to **Menu: Analyses à Choose…** or use the '**Choose Analyses**' button on the right-side of the ADE window (top button). The 'Choosing Analyses – ADE L' form will open. From the form, select the '**dc**' option button. Check the 'Save DC Operating Point'. We wish to sweep the input voltage (vds) of the NMOS device from 0V to 2.5V while observing the drain current (IDS). We will use 1001 steps in the sweep to give the output waveform sufficient resolution.

In the choosing analyses form. Check the 'Save DC Operating Point' box. Check the 'Design Variable' box as the sweep variable. Type *vds* as the variable name. Choose the 'Start-Stop' option button with Start = 0 and Stop = 2.5 for the sweep range. Select 'Linear' from the drop down menu as the sweep type. Choose the 'Number of Steps' option button with 1001 steps in the sweep. Ensure that the analysis is enabled by checking the 'Enabled' box near the bottom of the form. See Figure 6. OK the form. The **dc** analysis should now appear in the 'Analyses' list of the ADE window.



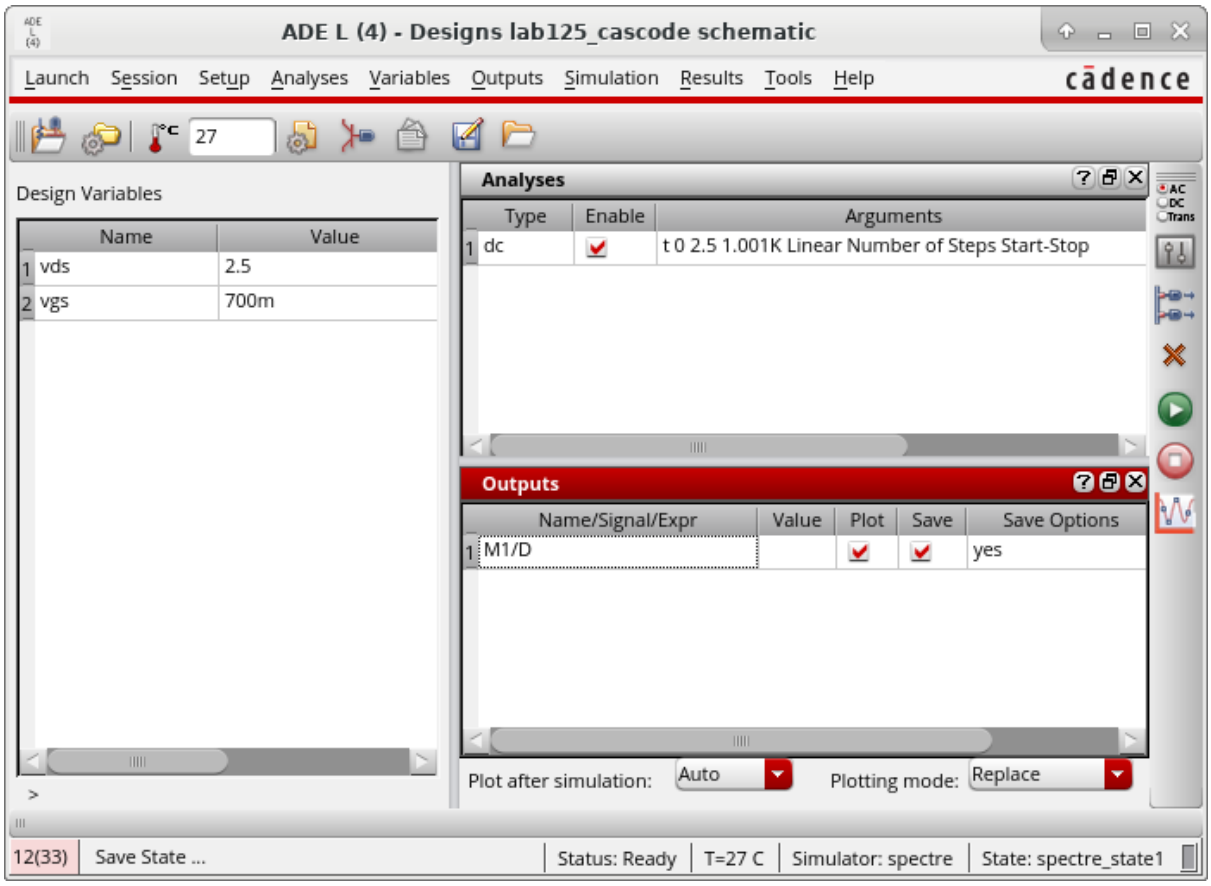
< Figure 122. The Choosing Analyses Form. >

(4.2.7) Select the drain node current (IDS) for plotting. In the Analog Design Environment window, select **Menu: Outputs à To Be Plotted à Select On Schematic** to enter the plotting mode of the ADE**.** The schematic window will automatically become current. Select only the drain node of the NMOS device. **Press the 'ESC' key to exit the plotting mode of the ADE**.

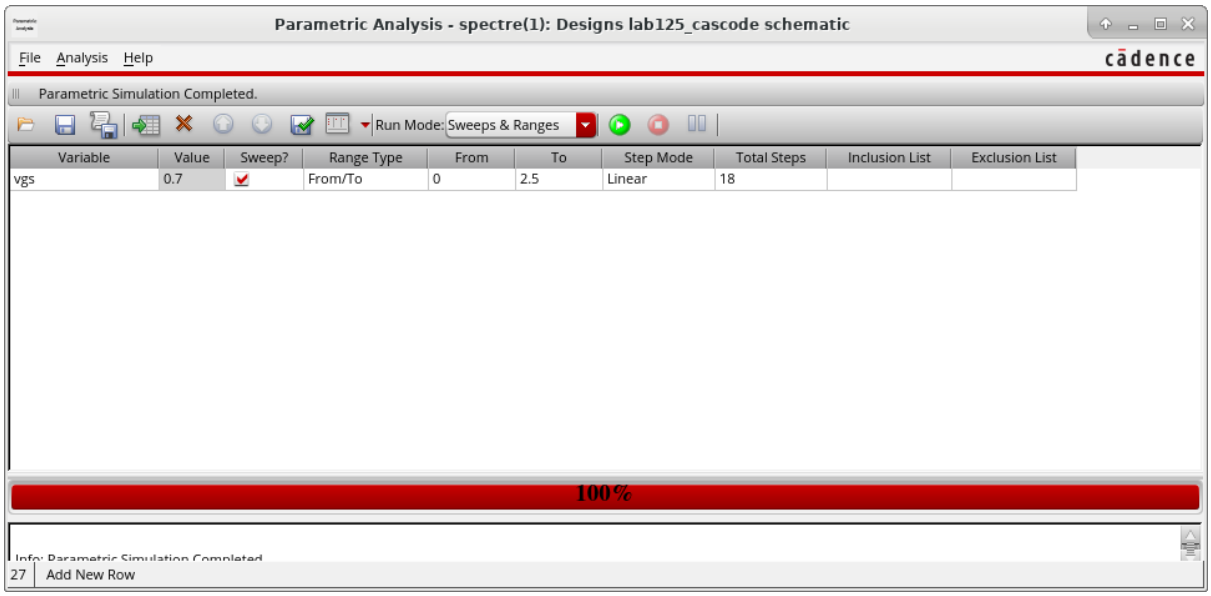
*Recall**,** if you want to plot a particular voltage, you click on the net/wire. If you want to save and plot the currents, you have to click a terminal **node** of a component (not the net/wire).*

(4.2.6) To sweep *vgs* we will use the parametric analysis tool. From the ADE window, choose **Menu: Tools → Parametric Analysis…** to invoke the Parametric Analysis tool. See Figure 8. In the Parametric Analysis window, select Sweeps and Ranges from the '**Run Mode'** drop down menu in the tool bar. In the '**Variable'** column, you should get a drop-down menu of the available design variables. Select vgs as the parametric sweep variable. Note, only the design variables in the 'Design Variables' section of the ADE will be available in the drop-down list along with the system variable temp. So, make sure you have copied the design variables from the schematic into the ADE using the Menu: Variables → Copy From Cellview… command.

In the '**Range Type**' drop down menu select 'From/To'. In the 'From' column type 0 as the start value of the parametric sweep, and in the 'To' column type 2.5 as the stop value of the parametric sweep. In the '**Step Mode**' column select 'linear' and in the **Total Steps** type 18.



< Figure 123. The ADE with the Design Variables, Analyses and Outputs. >



< Figure 124. Parametric Analysis Tool. Set up for a linear sweep of VGS → 0V to 2.5V in 18 steps. This will run the swept DC-analysis (dc) 18 times to create a family of curves. >

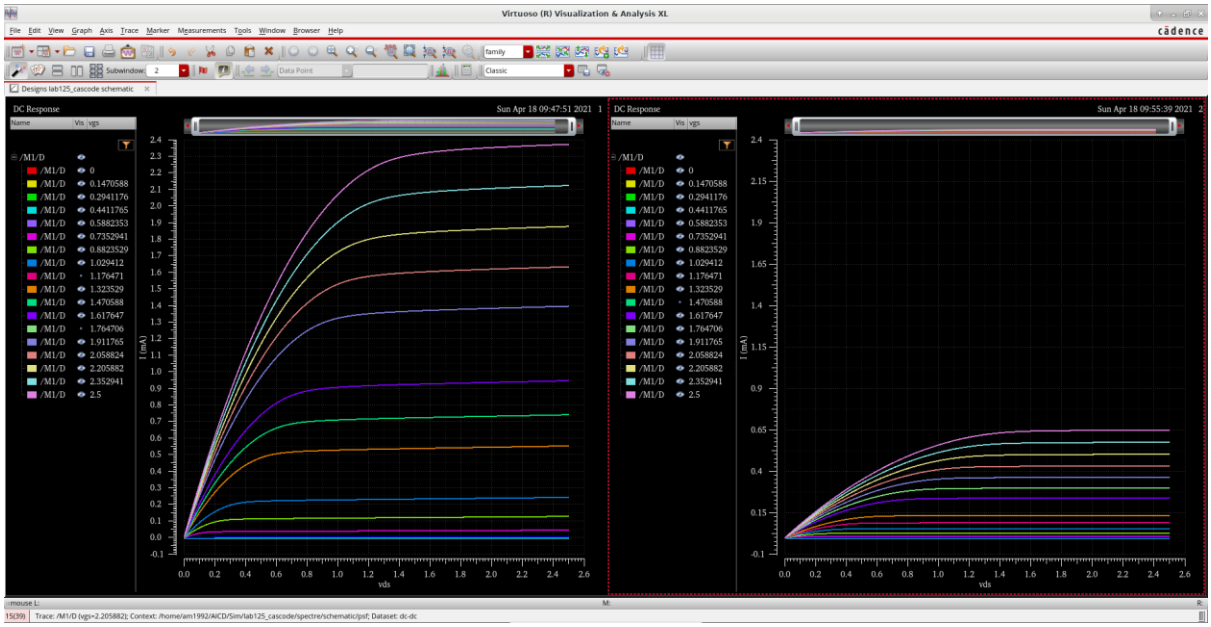
Note: The parametric tool works by repeating an analysis/simulation a specified number of times. While the parametric analysis tool allows us to specify additional variables to sweep, it is typically used to vary a parameter using far fewer sweep points than a DC-sweep, as it is much slower. The parametric analysis will run an analysis in the Analyses section of the ADE, once for each step of a parameter variation. A swept DC-analysis can efficiently handle sweeps of tens of thousands of sweep points. For the parametric tool, sweeps are best kept below 100 sweep points. A parametric sweep can be specified as a range to sweep or as an explicit list/set of values over which to iterate. (Run Mode: Sweeps and Ranges, or Parametric Set).

(4.2.8) The parametric analysis is run from the parametric analysis window not from the ADE window. In the Parametric Analysis window, run the '**Run Selected Sweeps**' command by clicking on the green play button. The simulation should run properly and both the text output window as well as the waveform output window should open automatically. With the above settings, the ADE will perform a swept-DC analysis of V_{DS} from 0V to 2.5V, a total of 18 times – one simulation for each value of V_{GS} (V_{GS} will be linearly incremented from 0V to 2.5V with each increment of V_{GS} given by 0.134V (=2.5V/18) as specified in the parametric form). See Figure 9 (left, reproduced in Figure 10). **Leave the waveform output window open.**

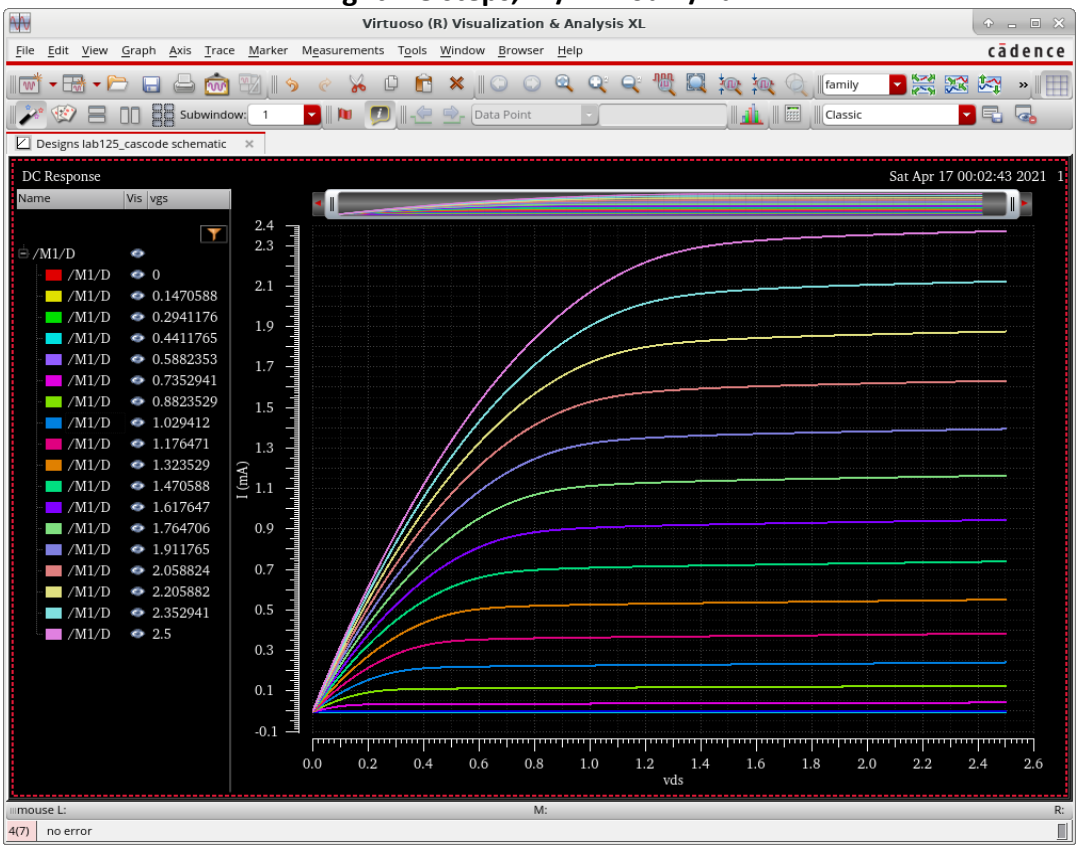
4.3 Increasing the Channel Length to Increase Output Impedance

(4.3.1) We will re-run the parametric simulation with the MOSFET channel length increased from 1um to 4um. Dimension the NMOS device 10u/4u (**width** = *10u*, **length** = 4u). Leave all other attributes unchanged.

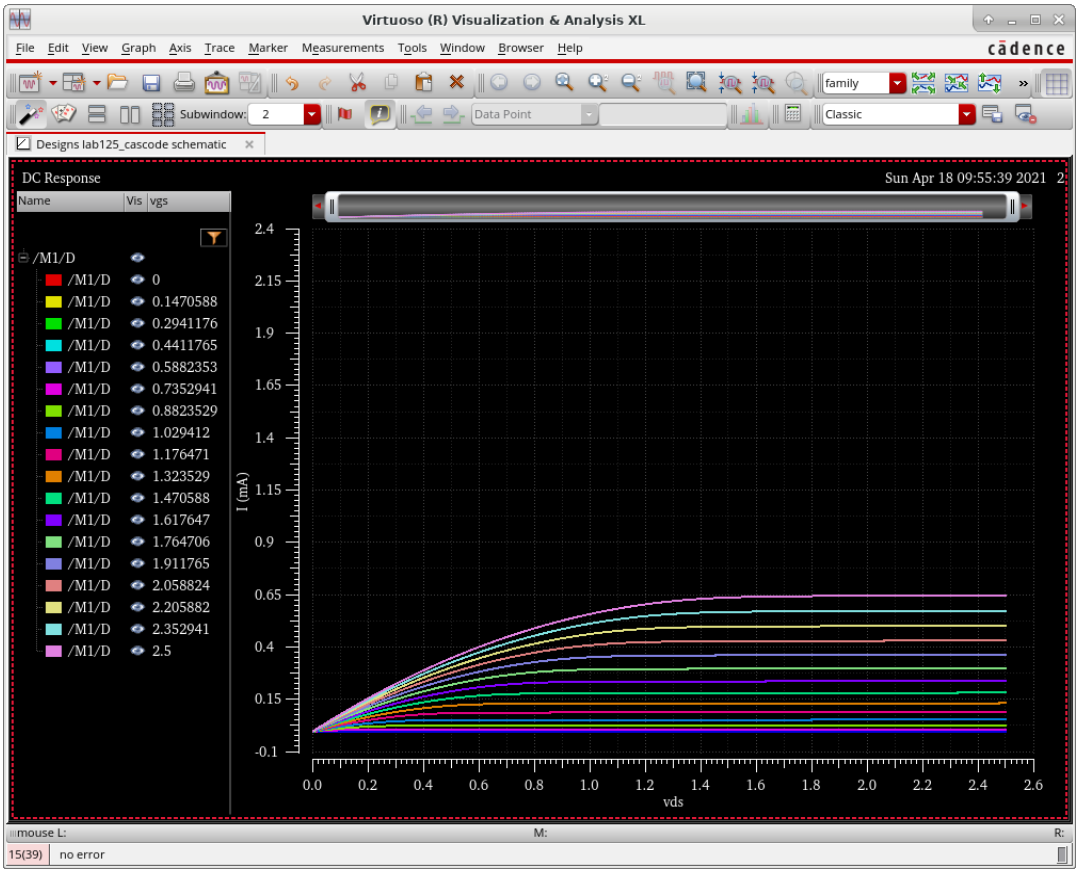
(4.3.2) Set the plotting mode of the ADE (lower right corner) to **New Sub-Window**. Re-run the parametric analysis; in the Parametric Analysis window, run the '**Run Selected Sweeps**' command by clicking on the green play button. The simulation should run properly and both the text output window as well as the waveform output window should open automatically. See Figure 9 (right, reproduced in Figure 11). These charts are on the same y-axis scale.



< Figure 125. Parametric DC-sweep of IDS vs. VDS vs. VGS.
Left: 18 steps, W/L = 10um/1um.
Right: 18 steps, W/L = 10um/4um.

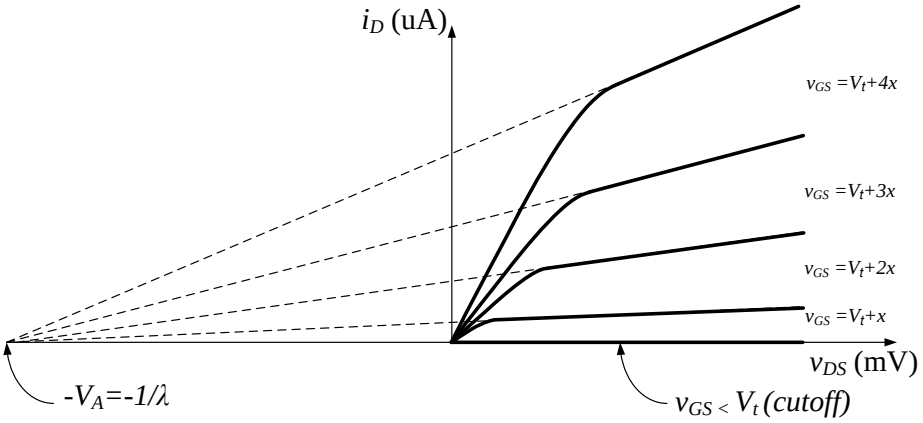


< Figure 126. Parametric DC-sweep of IDS vs. VDS vs. VGS. (18 steps, W/L = 10um/1um) >



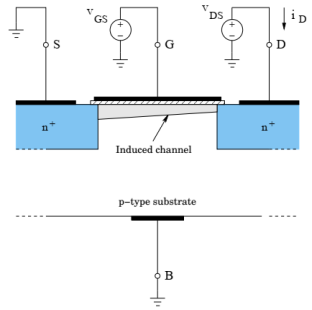
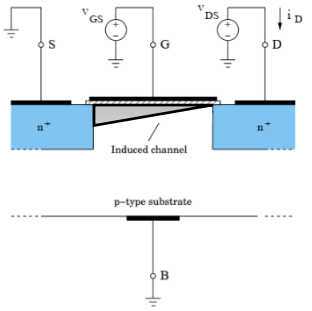
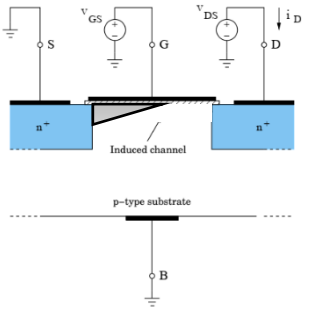
< Figure 127. Parametric DC-sweep of IDS vs. VDS vs. VGS. (18 steps, W/L = 10um/4um) >

(4.2.10) **Discussion**. From the slope of the large-signal I_{DS} vs. V_{DS} curves in the saturation region, we see that there is some dependency of I_{DS} on V_{DS} (as we simulated in labs 12.2 and 12.3). As we mentioned in this introduction, this arises due to **channel length modulation**.



< Dependency of I_{DS} on V_{DS} in the saturation region. >

In the triode region (1 in the Figure below), below pinch-off, the MOSFET channel is continuous from source-to-drain. As the MOSFET enters saturation (2 in the Figure below) where $V_{DS}(sat)=V_{GS}-V_T$, the MOSFET channel becomes pinched-off at the drain region. As the drain voltage V_{DS} increases beyond the pinch-off voltage (3 in the Figure below), the electric field pushes the channel pinch-off point away from the drain and down the channel, *effectively* making the drain-to-source channel length shorter.

1.	2.	3.
		
Below Pinch-Off	*At Pinch-Off*	*Above Pinch-Off*
$V_{DS} < V_{GS}-V_T$	$V_{DS}(sat)=V_{GS}-V_T$	$V_{DS} > V_{GS}-V_T$
$$i_{DS} = \frac{1}{2}k'\left(\frac{W}{L}\right)(V_{GS}-V_t)^2(1+\lambda V_{DS})$$ $$= \frac{1}{2}k'\left(\frac{W}{L}\right)(V_{GS}-V_t)^2\left(1+\frac{\Delta L}{L}\right)$$		

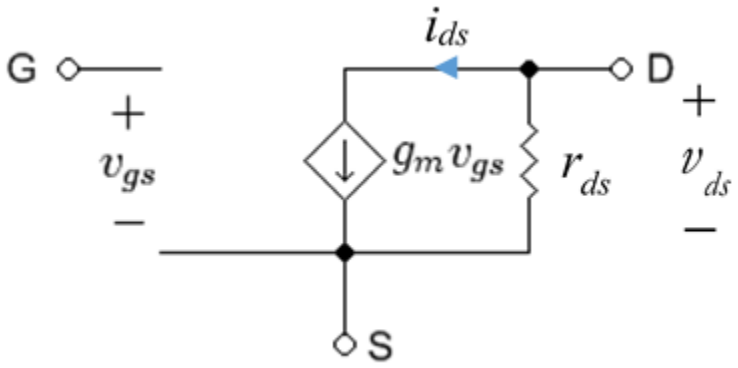
Since the drain current is inversely proportional to channel length L, I_{DS} increases with increasing V_{DS}. A variation of the drain voltage (in saturation) will therefore lead slight variation in drain current. Note that,

$$\lambda V_{DS} = \frac{\Delta L}{L}$$

where $\lambda = 1/(V_E L)$ is the channel length modulation parameter, a parameter that accounts for the change in I_{DS} with change in V_{DS}. This variation of I_{DS} with V_{DS} can be modelled as a resistance ***rds*** between the drain and source of the ideal MOSFET model. As we observed in lab 12.3, the effective resistance ***rds*** is given by,

$$r_{ds} = \left[\frac{\partial I_{DS}}{\partial V_{DS}}\right]^{-1} = \frac{V_A}{I_{DS}} = \frac{V_E L}{I_{DS}} = \frac{1}{I_{DS}\lambda}$$

The process parameter $\boldsymbol{V_E}$ is generally 5V/µmL for N-channel MOSFETs and 8V/µmL for P-channel MOSFETs. (For a summary of the phenomena refer to section 1.2.5 of the background material and section 4.2.3 in reference [3].)



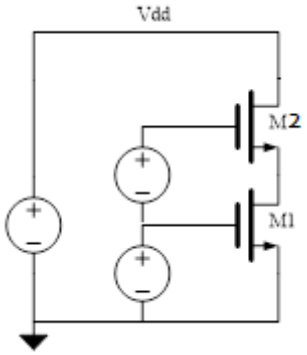
< Voltage controlled current source with (non-ideal) output impedance r_{ds}. >

r_{ds} lowers the voltage gain for a given transconductance as it appears in parallel with the load resistance (which is infinite for an ideal current-source load). We note from the above equation that r_{ds} is dependent on the channel length *L*. We can increase r_{ds} by increasing *L*. We saw this in the previous simulation when we repeated the parametric sweep with *L*=4um. The increased channel length makes the variation in effective length less sensitive as a function of overall length. However, we also see that by increasing L we also decrease the transconductance. To keep the transconductance broadly similar we therefore need to increase the width proportionally. This leads to a larger device with the accompanying trade-offs (namely, power and speed.)

4.3 Using a Cascode Device to Increase Output Impedance

(4.3.1) In this part of the lab, we will add a cascode stage to the NMOS testbench circuit in Figure 5. This is a commonly used circuit technique to mitigate the effects of channel length modulation and increase output impedance.

(4.3.2) Modify your existing testbench schematic by constructing a second testbench alongside your existing testbench circuit. The new circuit is the same as the previous one except for the addition of a cascode device. See Figure 12.



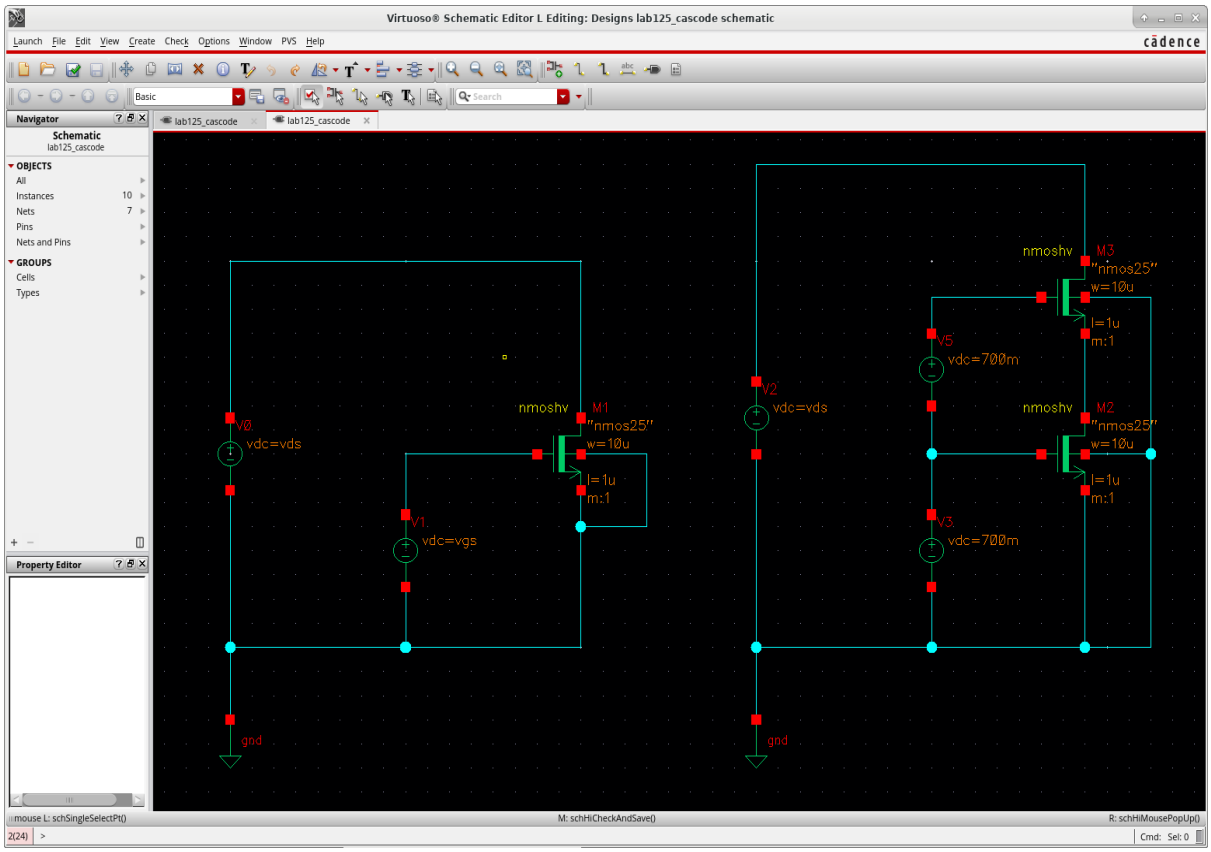
< Figure 128. Telescopic Cascode NMOS Testbench Circuit for Large Signal Transconductance Terminal Characteristics Evaluation. >

(4.3.3) Copying the circuit. Since the new circuit is similar to the previous one (except for the cascode device) you can save time/effort by copying part or all of the previous circuit. In the schematic window, on an empty part of the design canvas, left-click-and-hold the mouse button and drag a selection box around the circuit you want to duplicate. When you release the mouse button, the selected circuit will be highlighted pink/purple. Choose the **Copy** command (**Menu: Edit → Copy** or press '**c**') and click on a highlighted part of the circuit. A copy of the circuit should now attach itself to and move with the cursor. You can instantiate the copy by left-clicking anywhere on the design canvas.

(4.3.4) Modify you second circuit by adding a second NMOS FET device from the GPDK180 library (**nmoshv**). Add a telescopic repeated DC voltage source (**vdc**) for the additional NMOS gate-to-source bias.

(4.3.5) Set all NMOS devices (in both circuits) to a W/L ratio of 10um/1um (**Total Width** = *10u*, **Length** = *1u*). Set the gate-to-source voltages of all the NMOS devices to 0.7V (**DC Voltage** = *0.7*) – you can do this be setting the voltage attribute to 0.7V or by parametrising all vdc's to *vgs* and setting them to 0.7 in the ADE. Leave all other attributes/properties unchanged.

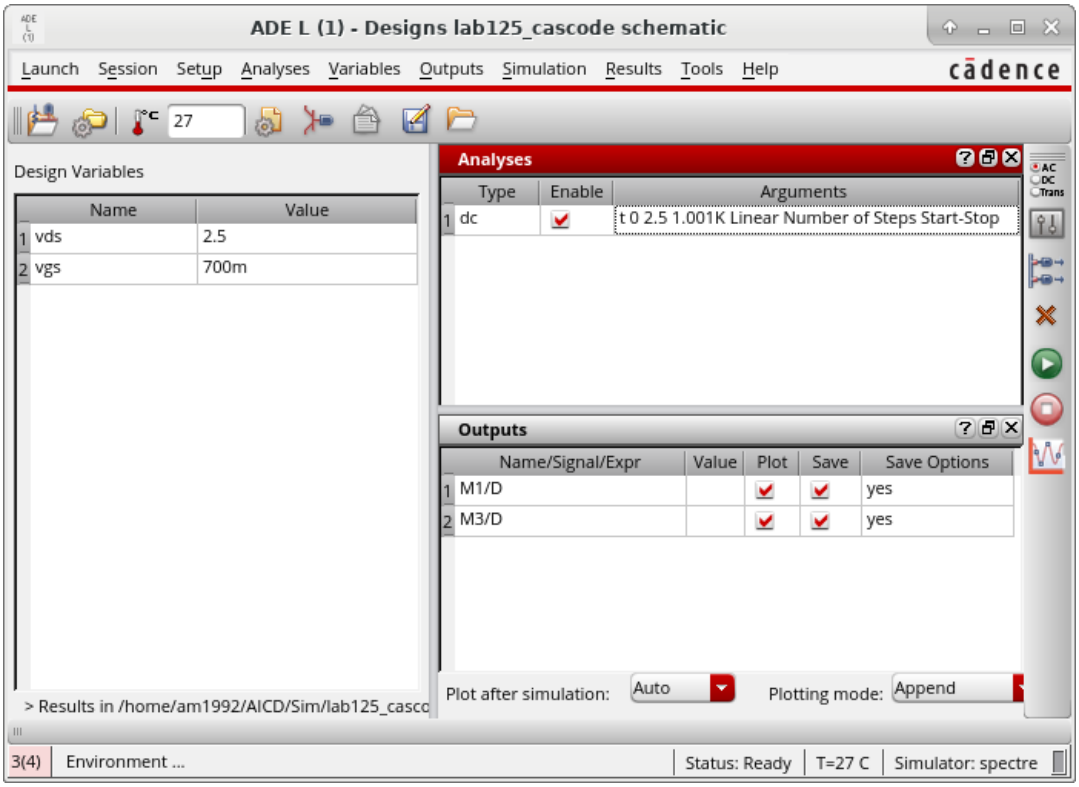
(4.3.6) Check and save your schematic using the **Check and Save** command.



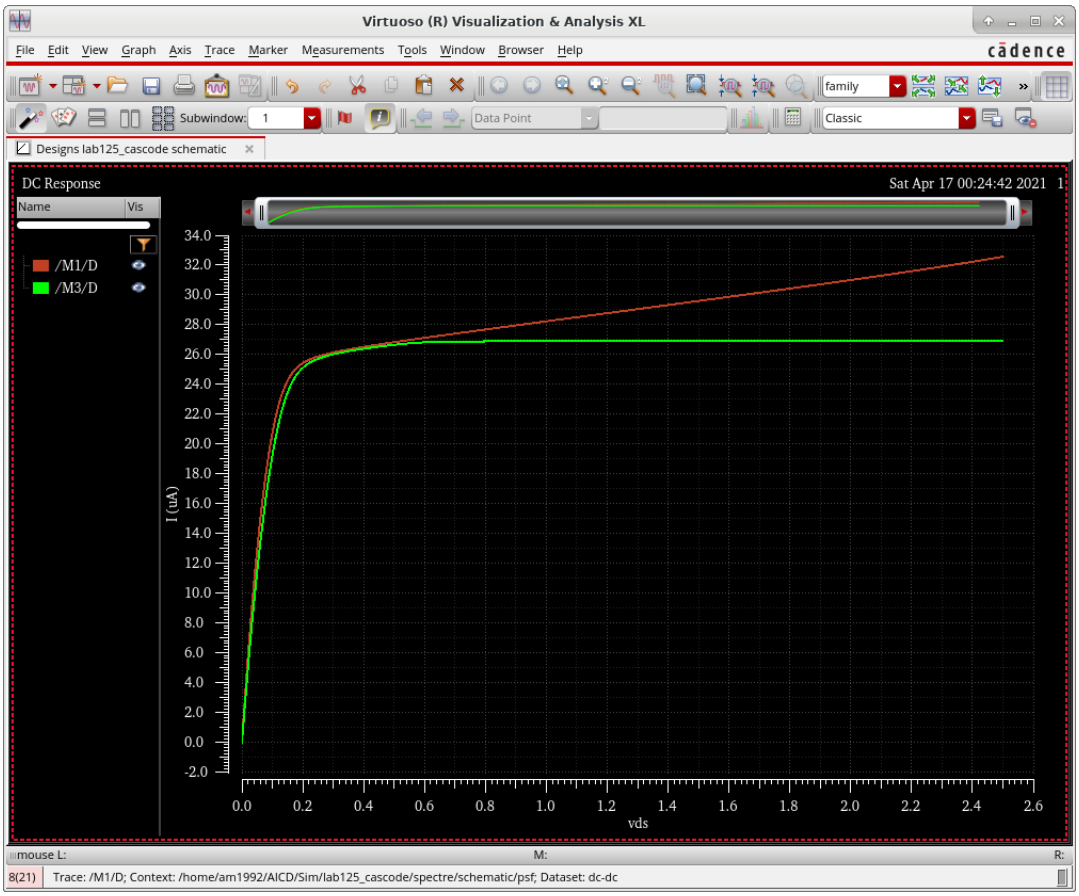
< Figure 129. Schematic with Non-Cascoded (left) and Cascoded (right) NMOS Testbench Circuit. >

(4.3.7) You should still have your ADE set up for simulation. (We will again sweep the input voltage *vds* of the NMOS device from 0V to 2.5V while observing the drain current IDS. We will use 1001 steps in the sweep to give the output waveform sufficient resolution.) If not, or you have created a new schematic, launch the ADE and follow the steps in the previous section to set up the swept-DC analysis (of VDS).

(4.3.8) For plotting, select the drain node current of the original testbench circuit (M1/D) and the drain node of the cascode device in the new testbench circuit (M3/D). **Press the 'ESC' key to exit the plotting mode of the ADE**. Run the '**Netlist and Run**' command by clicking on the green play button. The simulation should run without error and both the text output window as well as the waveform output window should open automatically. You should get a plot of I_{DS} vs. V_{DS} for the cascoded and non-cascoded NMOS devices. See Figure 15.



< Figure 130. The ADE with the Design Variables, Analyses and Outputs. >



< Figure 131. Plot of IDS vs. VDS for the Non-Cascoded (M1) and Cascoded Devices (M3).>

(4.3.9) **Discussion**. Consider the voltage at the drain of **M1** in the non-cascoded circuit (fig. 2). It is the variation of this voltage that modulates the length of the channel of **M1**. If this voltage is fixed, then no modulation occurs leading to the obviation of the need to consider the output resistance.

Now consider the voltage at the drain of **M2** in the cascoded circuit (fig. 12). This voltage is fixed by the required overdrive voltage in **M3** and (the fixed) V_{GSM3} forming a low impedance node, whilst the drain voltage of **M3** is free to vary as it is a high impedance node. Effectively, one may consider that **M3** is buffering the drain of **M2** from the high impedance node so that only a small amount of voltage variation is observed at this point. The low voltage variation leads to a low level of channel length modulation for a given process and device geometry. There will be some voltage variation as the current through the **M2**-**M3** cascode changes. It is left to the reader to consider the meaning of 'low impedance node' in terms of how the circuit driving the node may be modelled as a voltage source.

From Figure 12 with M2 and M3. Cascoding drastically lowers the V_{DS} variation seen by the transconductance device if both devices (transconductance and cascode) are in saturation. As a result, the variation in I_{DS} is reduced. The transconductance plus cascode pair will thus have a much greater *effective* r_{ds} and consequently a greater implicit small-signal voltage gain for a given device transconductance (g_m). Referring to section 6.8.1 in [3] shows that, the r_{ds} of the cascoded pair is equal to the r_{ds} of the common source device (M2, r_{ds2}) multiplied by the small-signal intrinsic gain of the cascode device (M3, A_{O3}),

$$r_{ds} = r_{ds1} \cdot A_{O2} = r_{ds1} \cdot r_{ds2} \cdot g_{m2}$$

where the intrinsic gain of the cascode device M3 is the product of its transconductance and output resistance ($A_{O3} = r_{ds3} \cdot g_{m3}$).

5. Circuit Simulation Exercises: Part 2

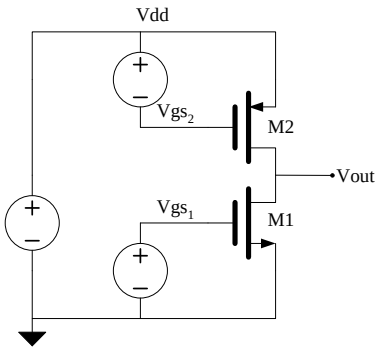
With guidance, you will:

- Simulate a telescopic cascode current-source loaded voltage amplifier circuit and compare its performance with that of a non-cascode equivalent.

5.1 The Current-Source Loaded Voltage Amplifier

(5.1.1) In this part of the lab, we will implement a current-source loaded single-stage NMOS FET voltage amplifier and observe the effect of cascoding on amplifier performance (gain).

(5.1.2) Create a new schematic for the current-source loaded voltage amplifier circuit as in Figure 16. Choose a meaningful cell name e.g., 'lab125_cascode_pt2'.



< Figure 132. Current-Source Loaded Voltage Amplifier
(Reproduced from Figure 16, section 4.2.2 of the background notes). >

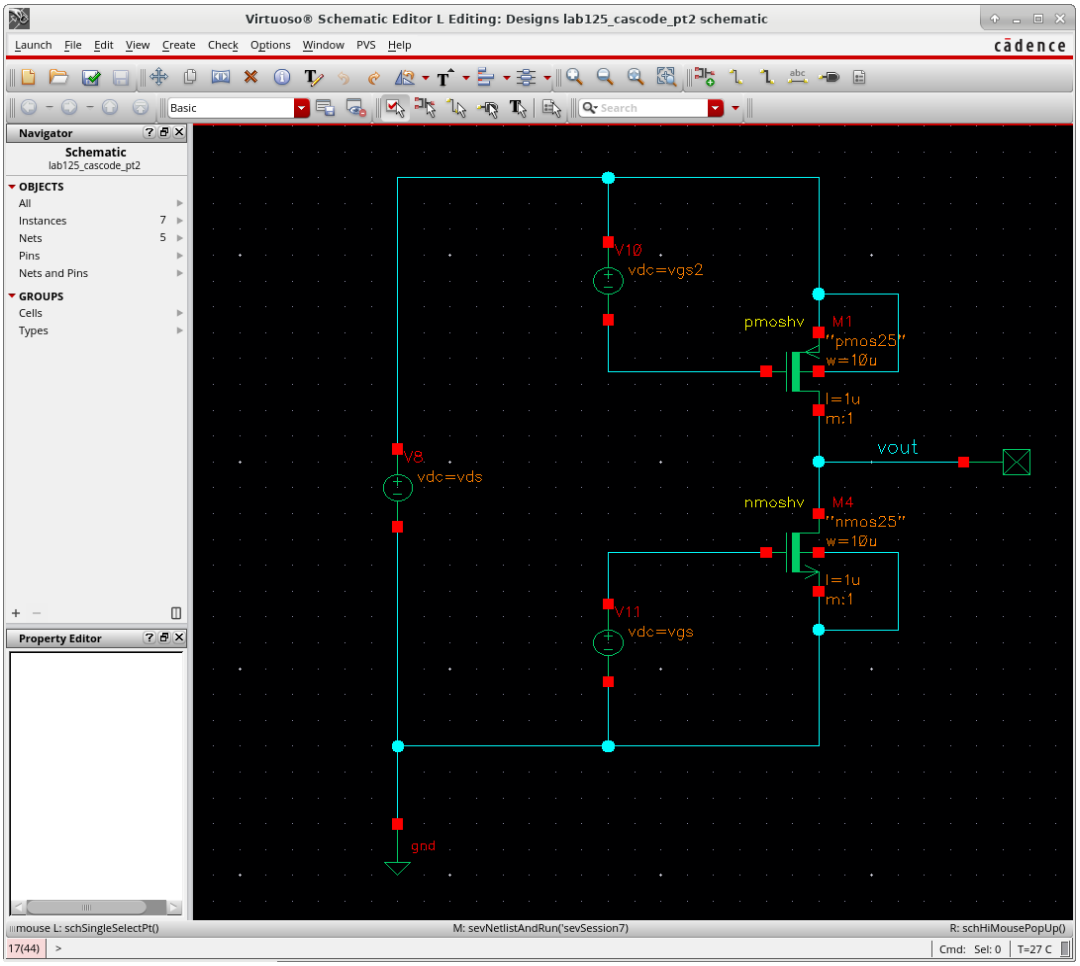
(5.1.3)) In your schematic editor window, create the circuit given in Figure 16. Instantiate three DC sources (**vdc**), a ground reference (**gnd**), and an NMOS FET and PMOS FET from the GPDK180 library (**nmoshv** and **pmoshv**).

Dimension both the NMOS and PMOS devices 10um/1um (**width** = *10u*, **length** = 1u). Leave all other attributes /properties unchanged.

Parameterise the DC supply voltage as *vds* (**DC Voltage** = vds), parameterise the gate-to-source voltage of the NMOS FET as vgs (**DC Voltage** = vgs), and parameterise the PMOS gate-to-source voltage as *vgs2* (**DC Voltage** = vgs2).

(5.1.4) Wire the components as in Figure 17 using the **Wire (narrow)** command. Name the output net *vout* using the **Create Wire Name** command. Use the **noConn** connector from the **basic** library to avoid floating-net warnings. Keep the schematic neat. See Figure 17.

(5.1.5) Check and save you schematic using the **Check and Save** command. It should check and save without any errors or warnings.



< Figure 133. Current-Source Loaded Voltage Amplifier. Variables for the DC supply voltage (vdc=vds) and the DC gate-to-source voltages (vdc=vgs, vdc=vgs2). >

5.2 Simulating the Voltage Amplifier using a Swept-DC Analysis

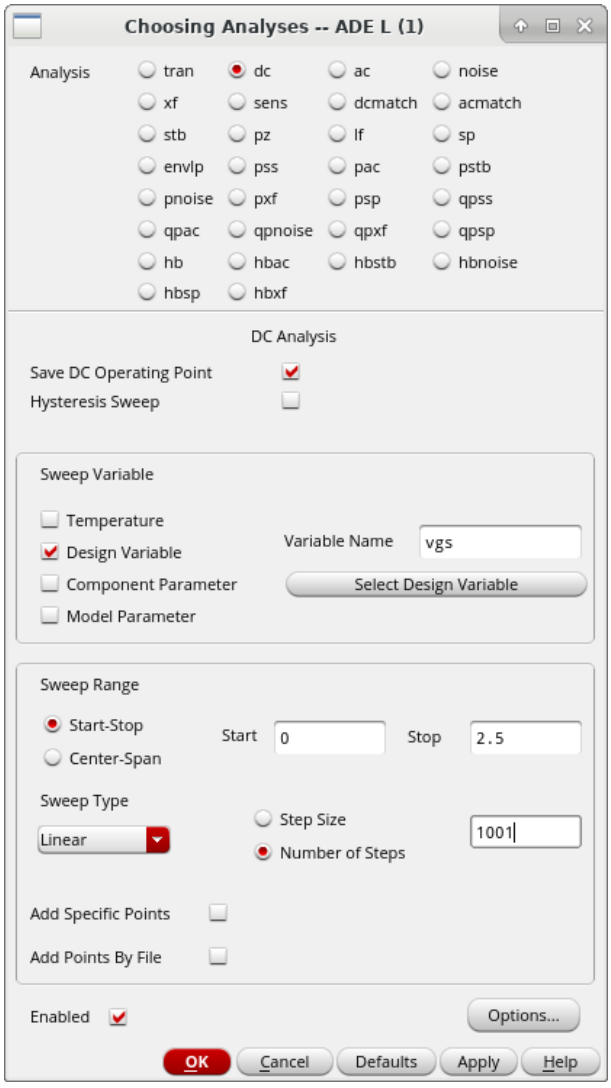
(5.2.1) We will simulate the basic voltage amplifier circuit using a DC-sweep of *vgs*. From the Schematic Editor window, launch the Virtuoso Analogue Design Environment (ADE) window from **Menu: Launch à ADE L**.

(5.2.2) Copy the design variables from the schematic into the ADE. For this circuit, these are the DC voltages from the three **vdc** components - vds, vgs, and vgs2. From the ADE window, choose **Menu: Variables à Copy From Cellview**. You should now see the vds and vgs variables

in the ‘Design Variables’ list of the ADE. Initialise vds = *2.5V*, vgs = *0.7V*, and vgs2 = *0.7V*. These values are used only to calculate the DC operating point of the amplifier circuit.

(5.2.3) From the ADE window, go to **Menu: Analyses à Choose…** or use the ‘**Choose Analyses**’ button on the right-side of the ADE window (top button). The ‘Choosing Analyses – ADE L’ form will open. From the form, select the ‘**dc**’ option button.

(5.2.4) We wish to sweep the input voltage (V_{gs}) of the NMOS device from 0V to 2.5V while observing the output voltage (V_{out}) – the input/output voltage transfer curve. We will use 1001 steps in the sweep to give the output waveform sufficient resolution.
In the choosing analyses form. Check the ‘Save DC Operating Point’ box. Check the ’Design Variable’ box as the sweep variable. Type ‘vgs’ as the variable name. Choose the ‘Start-Stop’ option button with Start = 0 and Stop = 2.5 for the sweep range. Select ‘Linear’ from the drop down menu as the sweep type. Choose the ‘Number of Steps’ option button with 1001 steps in the sweep. Ensure that the analysis is enabled by checking the ‘Enabled’ box near the bottom of the form. See Figure 17. OK the form. The **dc** analysis should now appear in the ‘Analyses’ list of the ADE window.

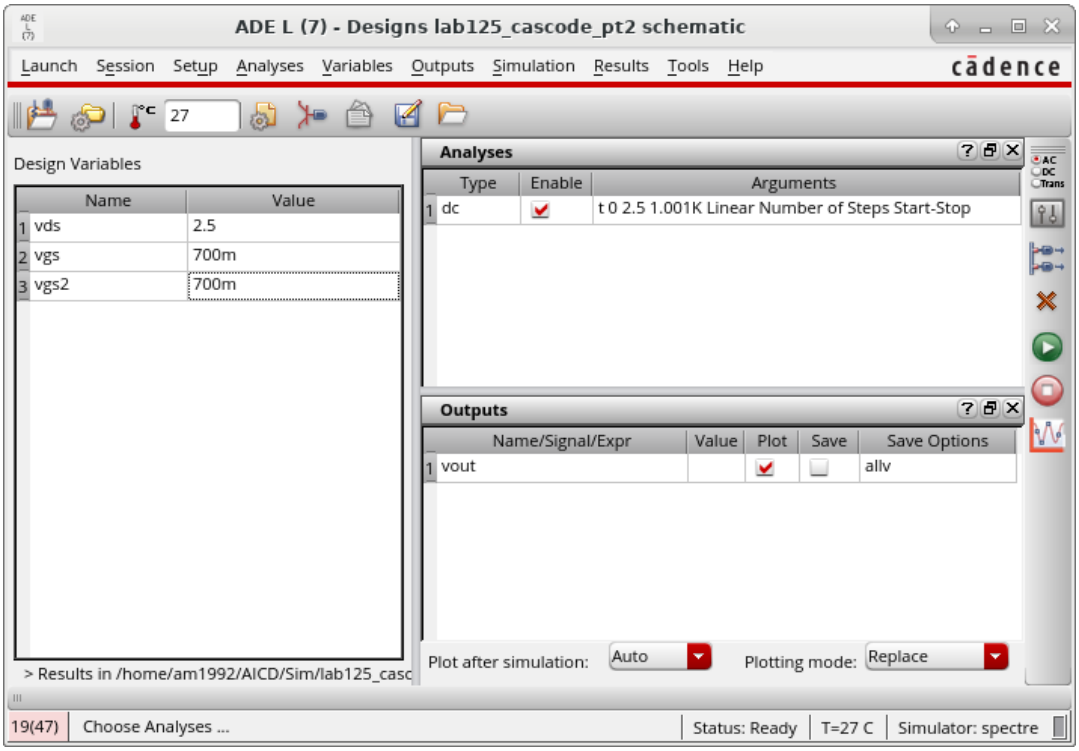


< Figure 134. The Choosing Analyses Form. >

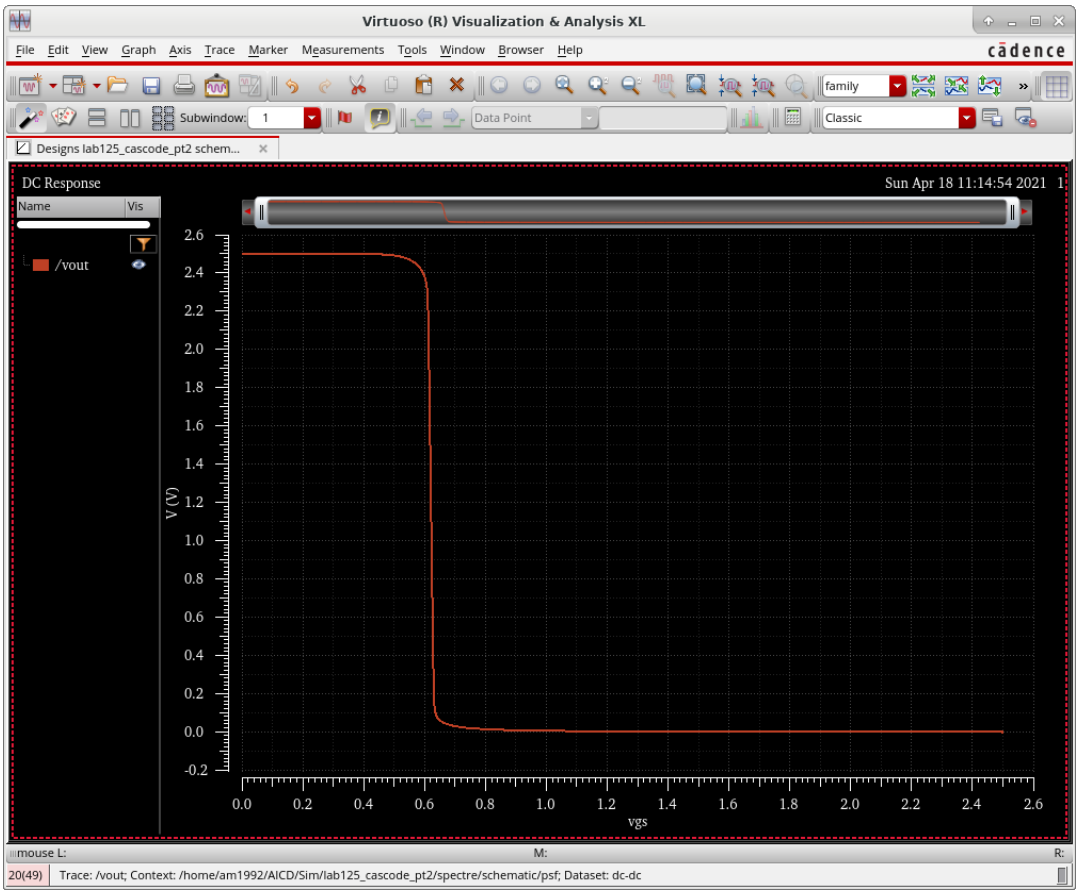
(5.2.5) Select the output net voltage (vout) for plotting. In the Analog Design Environment window, select **Menu: Outputs à To Be Plotted à Select On Schematic** to enter the plotting mode of the ADE**.** The schematic window will automatically become current. Select only the output net vout. **Press the 'ESC' key to exit the plotting mode of the ADE**.

(5.2.6) In the Analog Design Environment window, run the '**Netlist and Run**' command by clicking on the green play button. The simulation should run properly and both the text output window as well as the waveform output window should open automatically. See Figures 19 and 20.

(5.2.7) Repeat the simulation again with vgs swept between 0.5V and 0.7V for a better picture of the analogue region of the voltage transfer curve. See Figures 21 and 22.

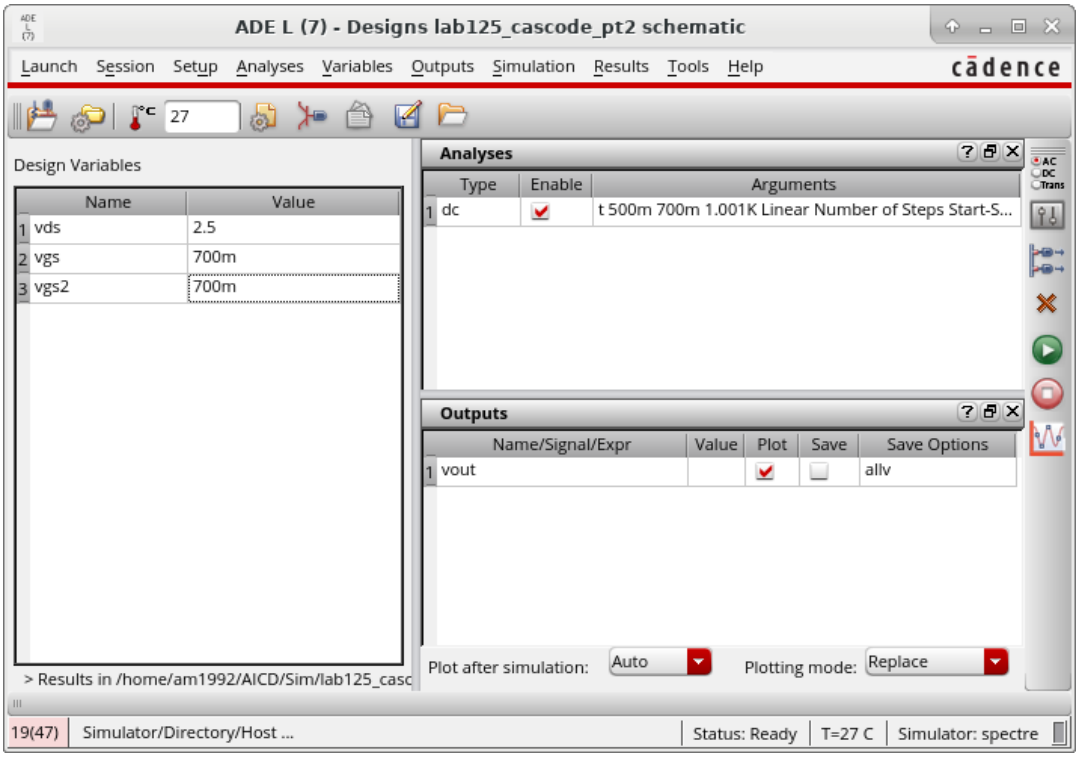


< Figure 135. The ADE with the Design Variables, Analyses and Outputs. >

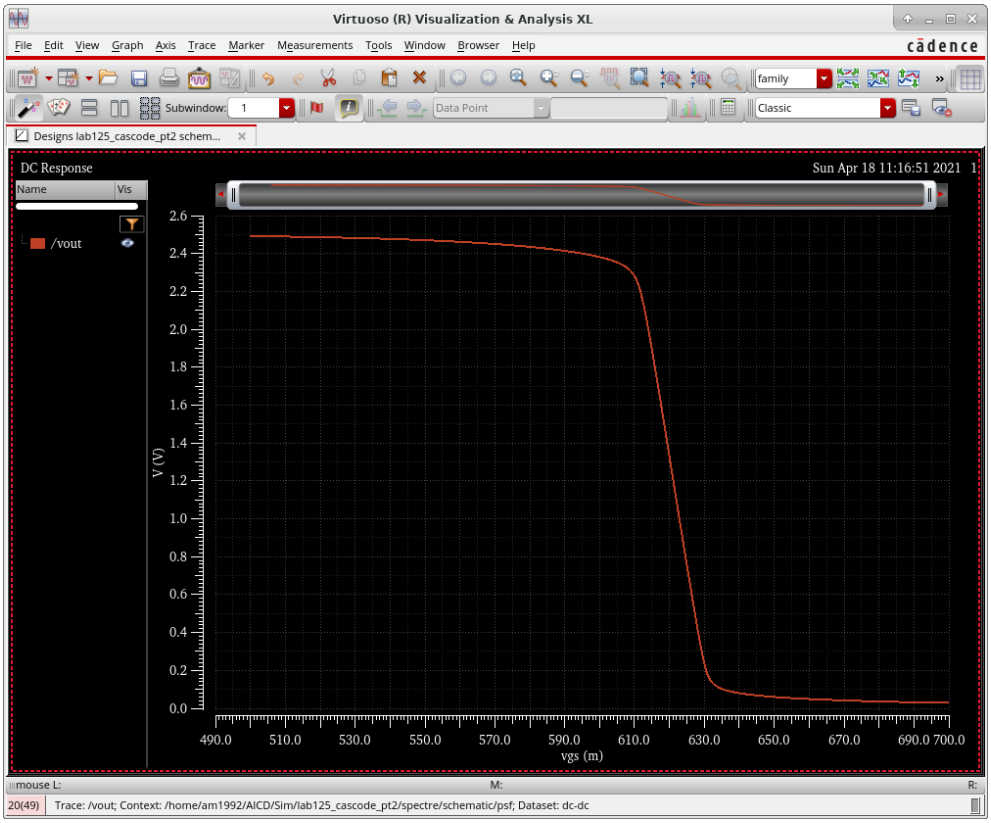


< Figure 136. Voltage Transfer Curve for the Current-Source Loaded Voltage Amplifier.

VDS=2.5V. VGS 0V → 2.5V. >



< Figure 137. The ADE with the Design Variables, Analyses and Outputs. >



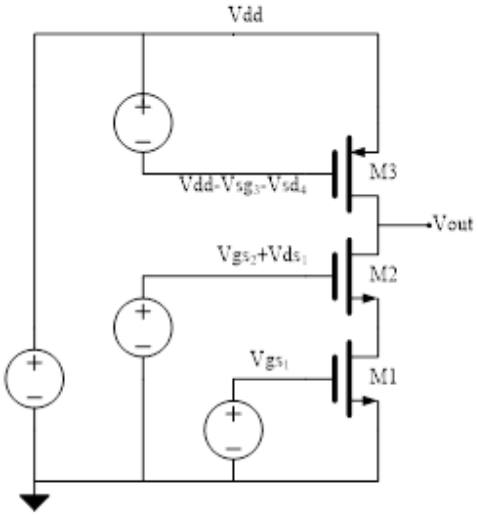
< Figure 138. Voltage Transfer Curve for the Current-Source Loaded Voltage Amplifier.

VDS=2.5V. VGS 0.5V → 0.7V. >

5.3 Cascoding the Transconductor Device to Increase Amplifier Gain

(5.3.1) We next will add a cascode stage to the NMOS transconductance device. This will increase the output impedance of the transconductance stage and consequently gain.

(5.3.2) Modify the schematic by constructing a second amplifier circuit alongside your existing amplifier circuit. The new circuit is the same as the previous one except for the addition of a cascode device in the transconductance stage. See Figure 23.



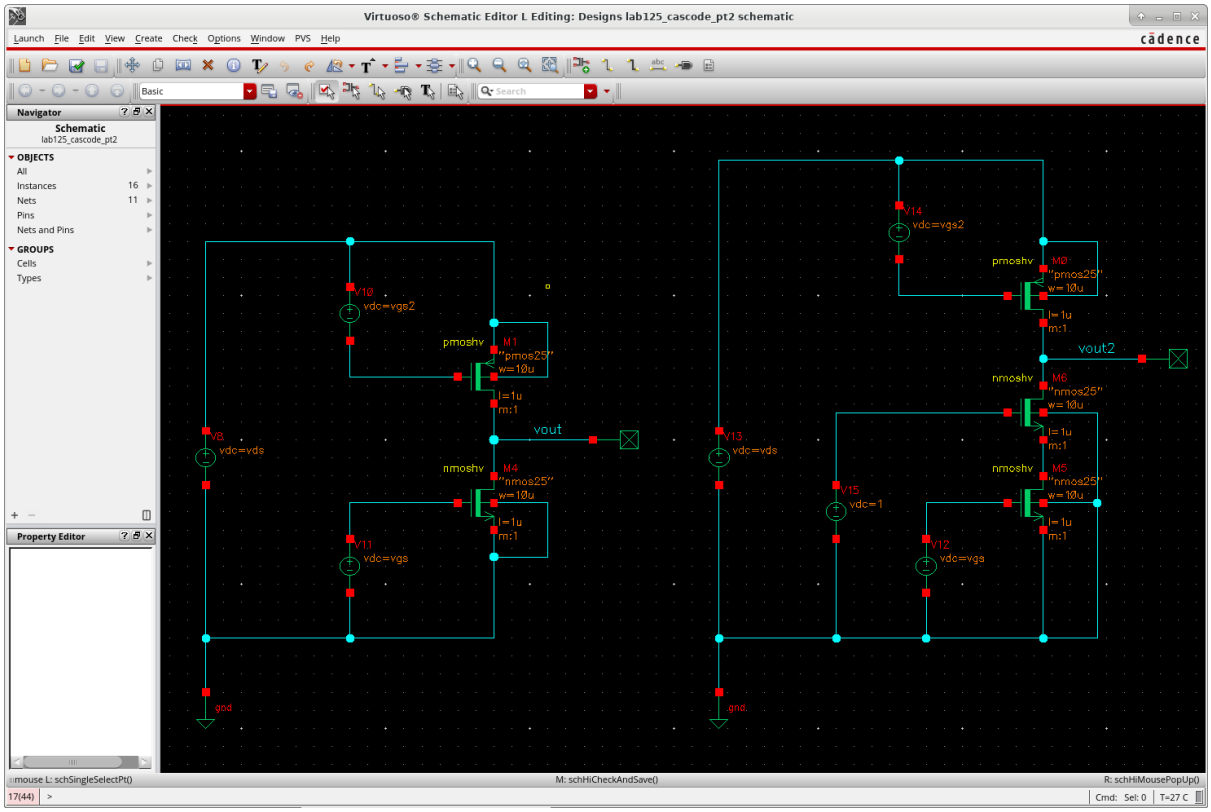
< Figure 139. Current-Source Loaded Voltage Amplifier with Cascoded Transconductance Stage>

(5.3.3) Copying the circuit. As before, in the schematic window, on an empty part of the design canvas, left-click-and-hold the mouse button and drag a selection box around the circuit you want to duplicate. Choose the **Copy** command (**Menu: Edit → Copy** or press '**c**') and click on a highlighted part of the circuit. A copy of the circuit should now attach itself to and move with the cursor. Instantiate the copy by left-clicking anywhere on the design canvas. See Figure 24.

(5.3.4) For the new circuit, all FET devices should be 10um/1um (**width** = *10u*, **length** = 1u). Add a separate DC voltage source (**vdc**) for the NMOS cascode device. Set the vdc for the cascode device to 1V (**DC Voltage** = *1*). Leave all other attributes/properties unchanged. See Figure 24. You can zoom in for closer inspection if needed.

(5.3.5) Change the name of the output net to vout2 (this avoids creating an implicit connection between the outputs of the two circuits.) Select the second output net voltage (vout2) for plotting alongside the first one (vout). **Press the 'ESC' key to exit the plotting mode of the ADE**.

(5.3.6) Check and save you schematic using the **Check and Save** command. It should check and save without any errors or warnings.

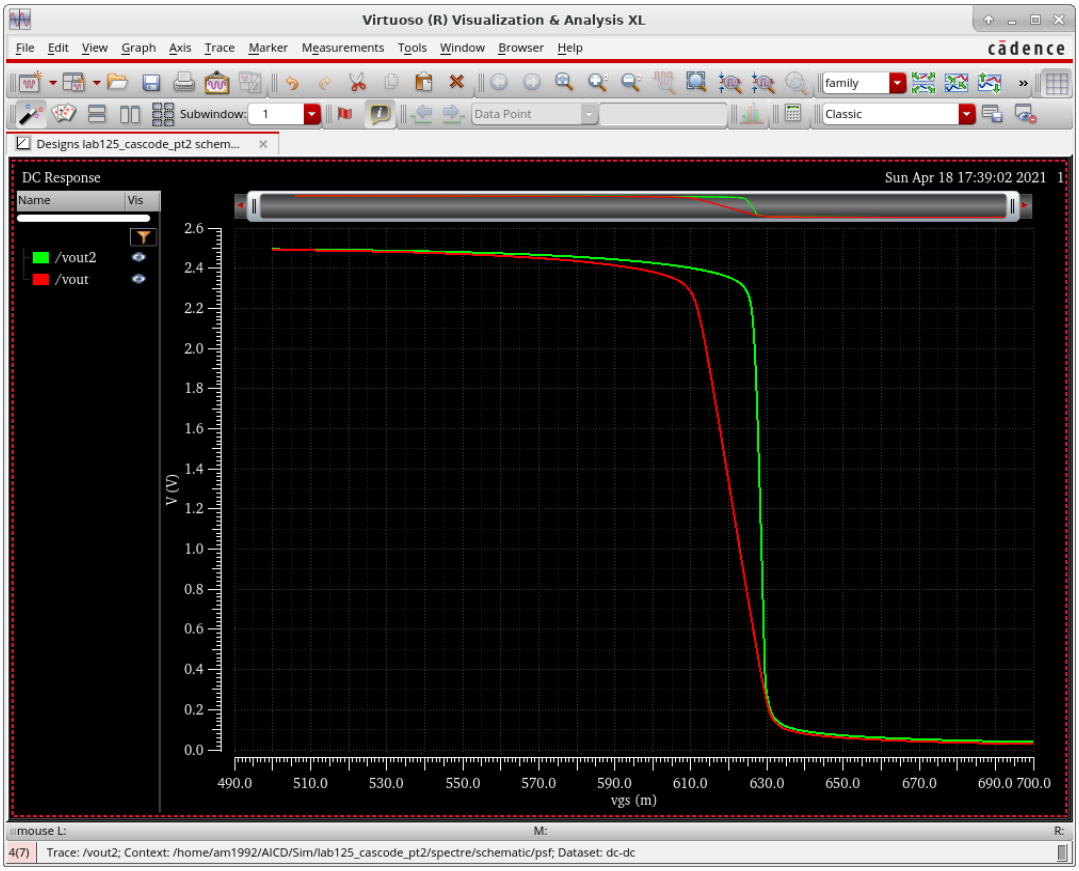


< Figure 140. Schematic with Non-Cascoded (left) and Cascoded (right) Current-Source Loaded Voltage Amplifiers. Only the Transconductance Stage is Cascoded. >

(5.3.6) From the Analog Design Environment window, re-run the DC sweep. Run the '**Netlist and Run**' command by clicking on the green play button. The simulation should run properly and both the text output window as well as the waveform output window should open automatically. See Figure 25.

(5.3.7) **Discussion**. Comparing the output of the non-cascoded (red) and cascoded (green) stages, we see the increased voltage gain of the latter (indicated by the steeper slope of the green VTC curve) is due to the higher effective output resistance of the transconductance stage. In this case, the output impedance of the non-cascoded stage is given by ($r_{op}||r_{on}$) and for the cascoded stage it is given by ($r_{op}||r_{on}.r_{on}.g_{mn}$), where r_{op} is the output resistance of the PMOS load device and r_{on} is the output resistance of the NMOS transconductance and identical cascode devices. g_{mn} is the transconductance of the cascode device. $A_{on}=r_{on}.g_{mn}$.

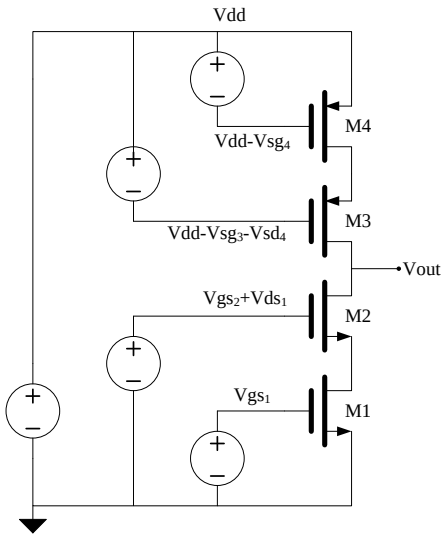
We will also note (briefly at this stage) that with the increased gain, comes the need to accurately set the DC operating point (the steeper the slope of the VTC the more sensitive the amplifier is to imprecise biasing). This cannot be done by designing for nominal device parameters in the same way we may design discrete circuits; instead, we must use feedback to set the DC operating point. We will discuss this important topic in the coming labs.



< Figure 141. Voltage Transfer Curves for the Non-Cascoded and Cascoded Current-Source Loaded Voltage Amplifiers. VDS=2.5V. VGS 0.5V → 0.7V. >

5.4 Cascoding the Transconductor and Load Devices to Increase Amplifier Gain

(5.4.1) Finally, we will add a second cascode stage, this time for the load device. Cascoding just the transconductance device makes the load device the weak link. Cascoding both the transconductor and load will maximise the gain.

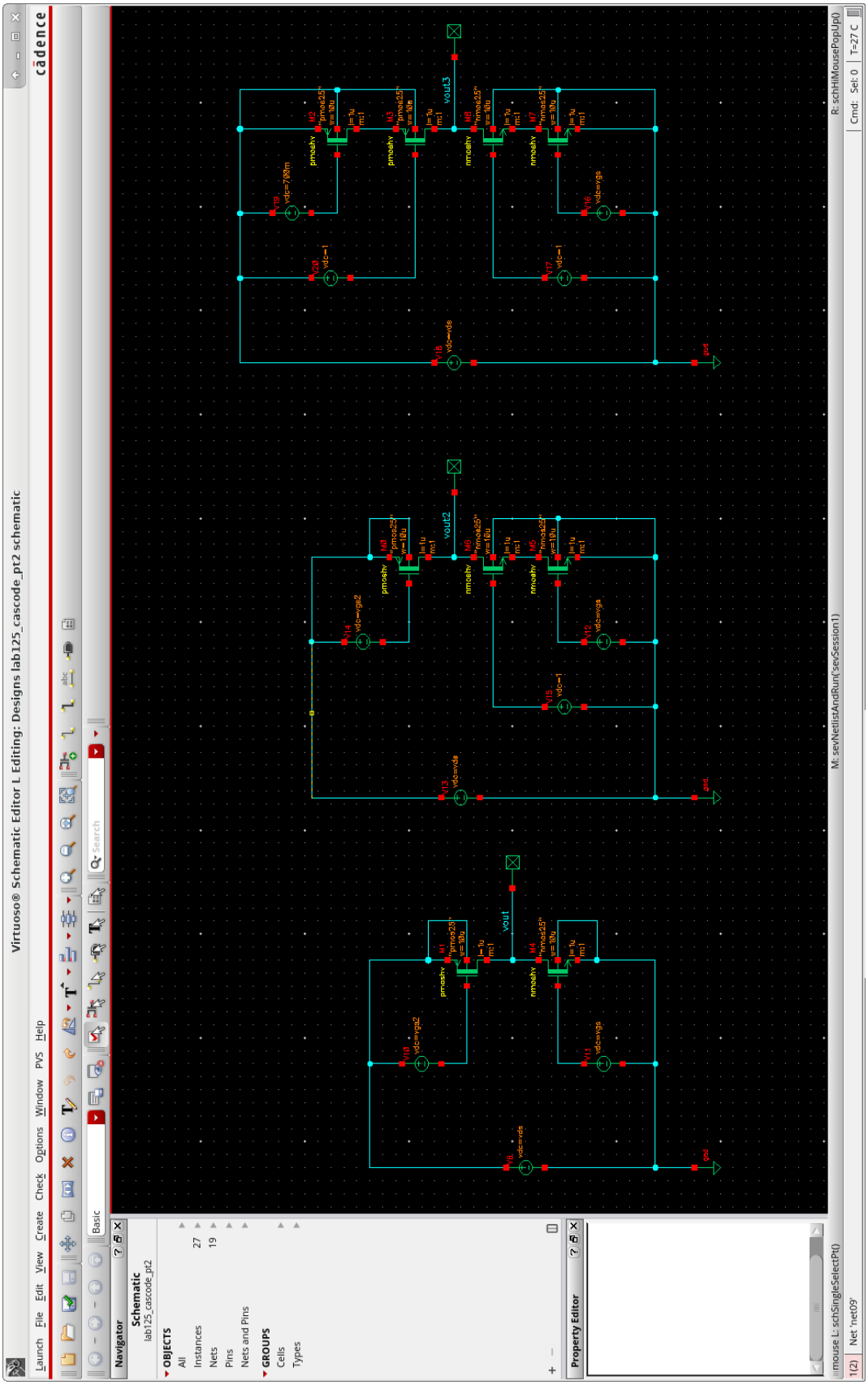


< Figure 26. Cascode NMOS Characteristics Evaluation. Reproduced from Figure 14, section 4.1 of the background material. >

(5.4.2) Again, for the new circuit, all FET devices should be 10um/1um (**width** = *10u*, **length** = 1u). Add another separate DC voltage source (**vdc**) for the PMOS cascode device. Set the vdc for the cascode device to 1V (**DC Voltage** = 1). Leave all other attributes/properties unchanged. See Figure 27.

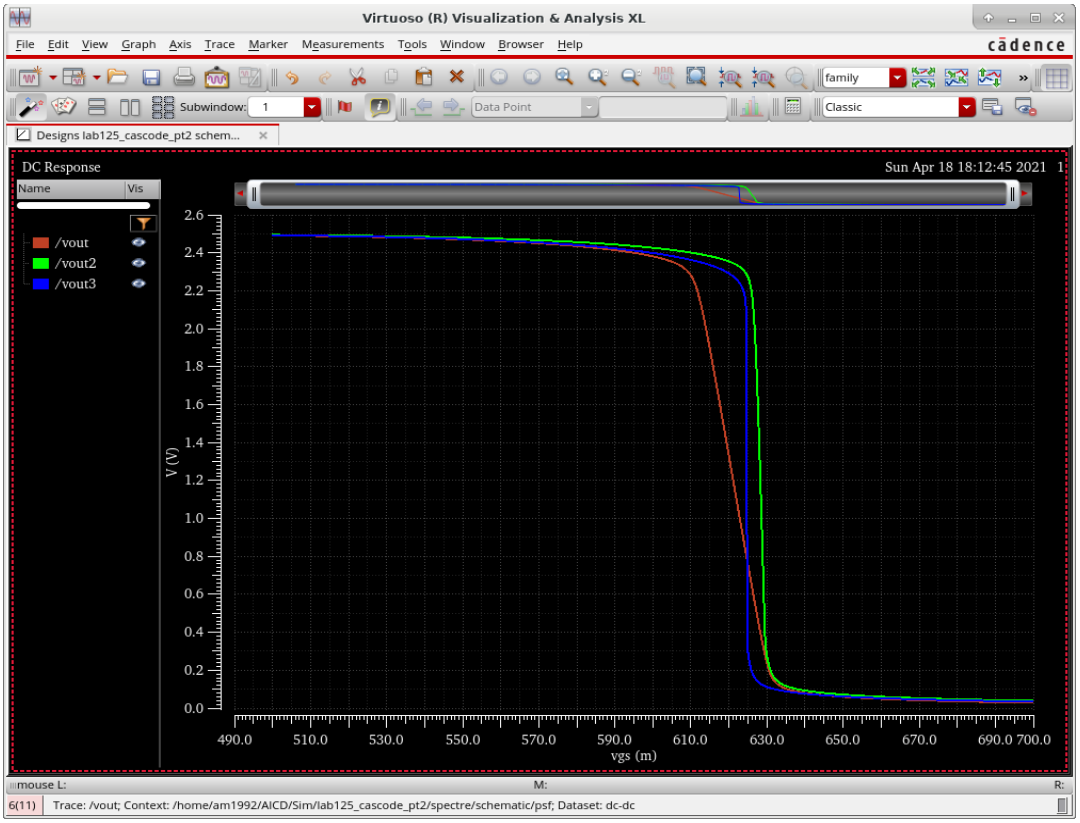
(5.4.3) Change the name of the output net to vout3. Select the third output net voltage (vout3) for plotting alongside the previous circuits. **Press the 'ESC' key to exit the plotting mode of the ADE**.

(5.4.4) Repeat the previous steps to add a third amplifier circuit alongside your existing circuits. You should now have three versions of the current-source loaded amplifier circuit: (1) an amplifier with no cascoding, (2) an amplifier with only the transconductance stage cascoded, and (3) an amplifier both the transconductance and load devices cascoded. See Figure 27.

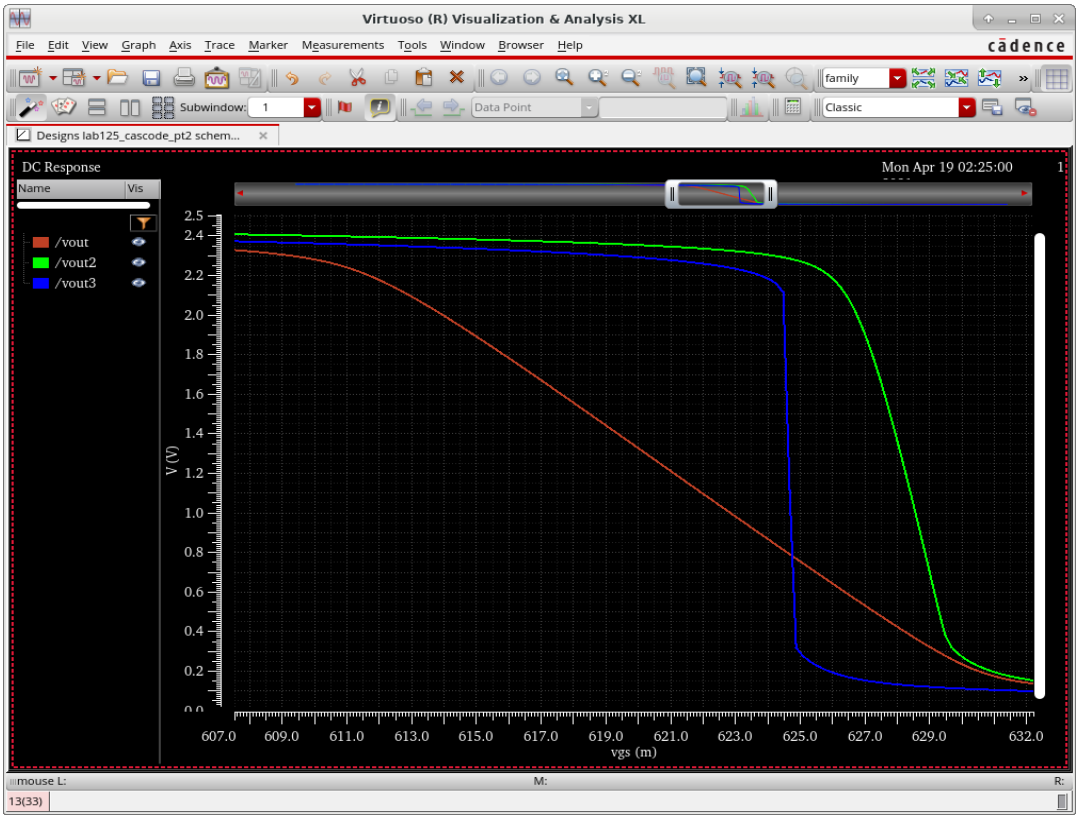


< Figure 142. Schematic with Non-Cascoded (left), Transconductor-Only Cascoded (middle) and Fully Cascoded (right) Current-Source Loaded Voltage Amplifier. >

(5.4.5) From the Analog Design Environment window, re-run the DC sweep. Run the '**Netlist and Run**' command by clicking on the green play button. The simulation should run properly and both the text output window as well as the waveform output window should open automatically. See Figures 28 and 29.



< Figure 143. Voltage Transfer Curves for the Non-Cascoded (red), Transconductor-Only Cascoded (green), and Fully Cascoded Current-Source Loaded Voltage Amplifiers. VDS=2.5V. VGS 0.5V → 0.7V. >



< Figure 144. Close-Up of Figure 28 in the Analogue Region. >

(5.4.6) **Discussion**. Again, we see the increased voltage gain of the fully cascoded amplifier (blue) is due to the higher total output resistance $(r_{op}.r_{op}.g_{mp})||(r_{on}.r_{on}.g_{mn})$.

STUDENT TASK: Given the need for accurate/precise biasing (as discussed above), keep vgs=700mv and perform an **ac** simulation to establish that there is no gain (ac 1k-1M). establish accurate vgs-quiescent = 624.7mv – gain is now 15k (83dB)]

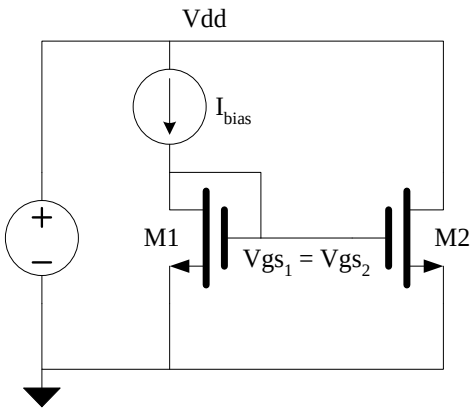
6. Circuit Simulation Exercises: Part 3

With guidance, you will:

- Investigate cascoding in a MOSFET current mirror.

6.1 A Basic Current Mirror

(6.1.1) Create a new schematic for the basic current mirror circuit in Figure 30. A summary of the circuit's operation is given in section 4.3.1 of the background material.



< Figure 145. Basic Current Mirror. Reproduced from Figure 18, section 4.3.1 of the background material. >

(6.1.2) From the library manager, invoke the New File form as you have done previously – ensuring you are in your Designs library – giving your new schematic a meaningful name such as 'lab125_cascode_pt3'.

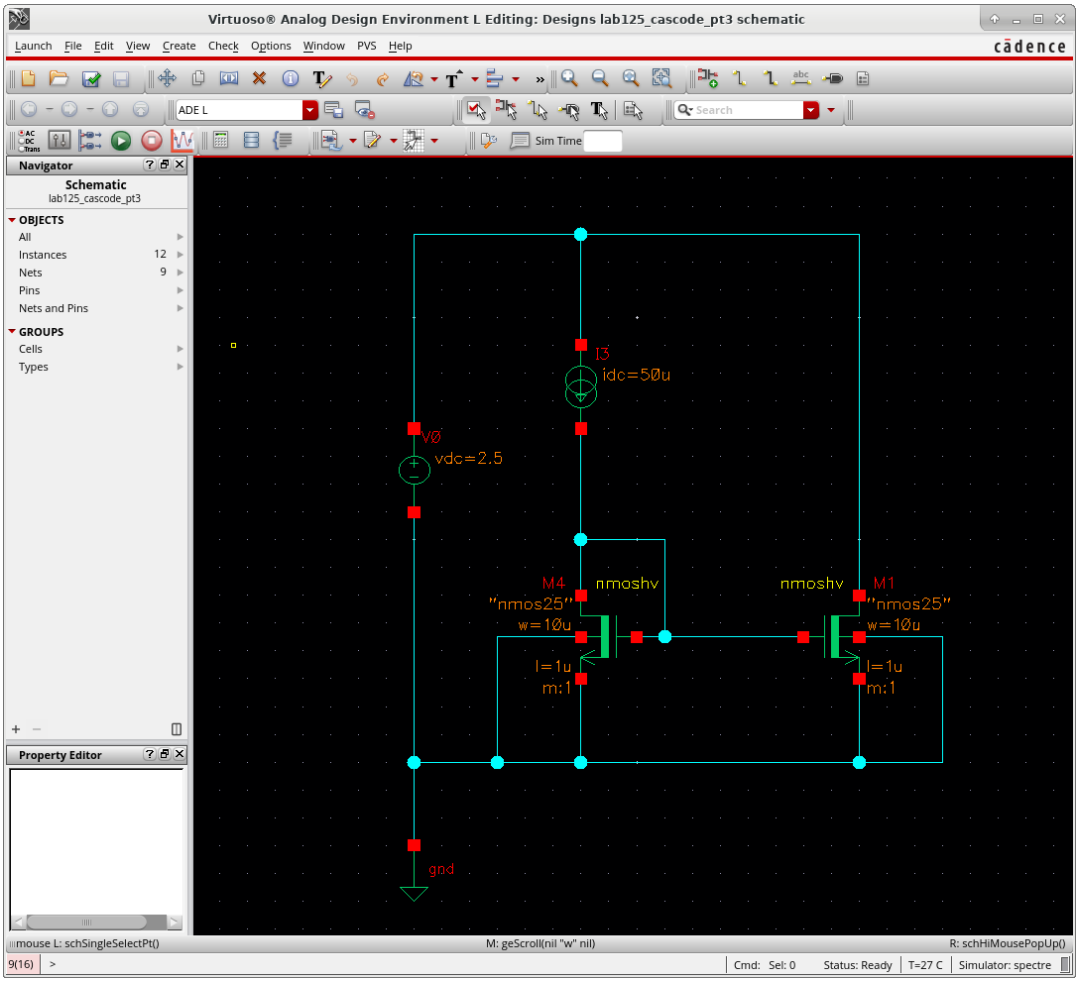
(6.1.3) In your schematic editor window, create the circuit given in Figure 30. Instantiate one ideal DC voltage source (**vdc**) and one ideal DC current source (**idc**) from the analogLib library. Instantiate two NMOS FETs from the GPDK180 library (**nmoshv**). Set the DC supply voltage to 2.5V (**DC Voltage** = 2.5), set the DC current reference to 50uA (**DC current** = 50u). Dimension both NMOS devices 10um/1um (**width** = *10u*, **length** = 1u). Leave all other attributes unchanged.

(6.1.4) Wire the components as in Figure 30 (31) using the **Wire (narrow)** command.

(6.1.6) Check and save your schematic using the **Check and Save** command. Remember to use the output area of the CIW to diagnose any problems with your schematic and/or simulations if there are any warning or error messages when checking, saving, or simulating your work.

6.2 Simulating the DC operating Point of the Current Mirror

(6.2.1) We will simulate the current mirror using a DC Analysis. From the Schematic Editor window, launch the Analogue Design Environment (ADE) from **Menu: Launch à ADE L**.

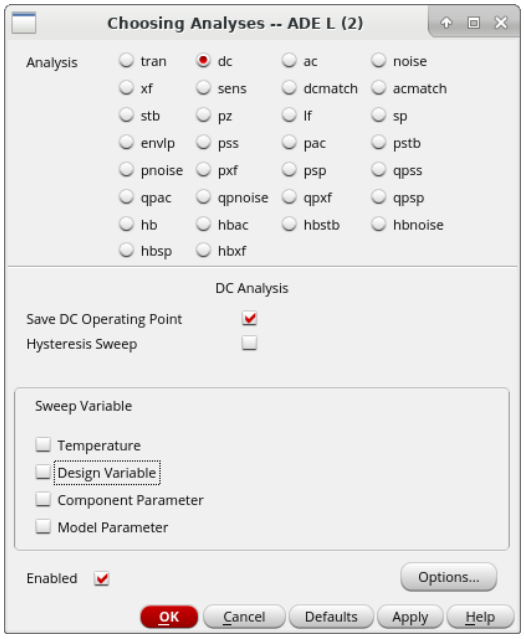


< Figure 146. Schematic of the Basic Current Mirror. 10um/1um. >

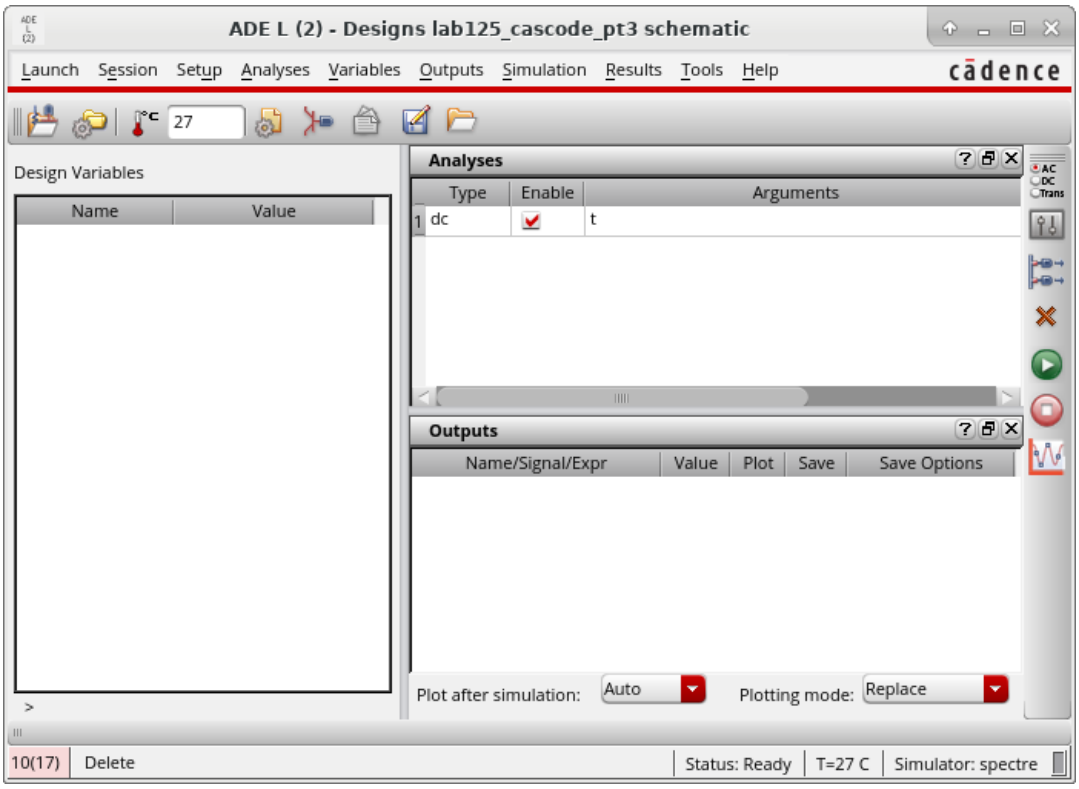
(6.2.2) From the ADE window, go to **Menu: Analyses à Choose…** or use the **Choose Analyses** button on the right-side of the ADE window (top button). The 'Choosing Analyses – ADE L' form will open. On the form, select the '**dc**' option button. Check the 'Save DC Operating Point' box. OK the form.

(6.2.3) At this stage we are only simulating for the DC operating point. Simulating the DC operating point does not produce an output waveform, so we do not need to select any outputs to be plotted.

(6.2.4) In the Analog Design Environment window, run the '**Netlist and Run**' command by clicking on the green play button. The simulation should run without error. The text output window should appear but there will be no waveform window.



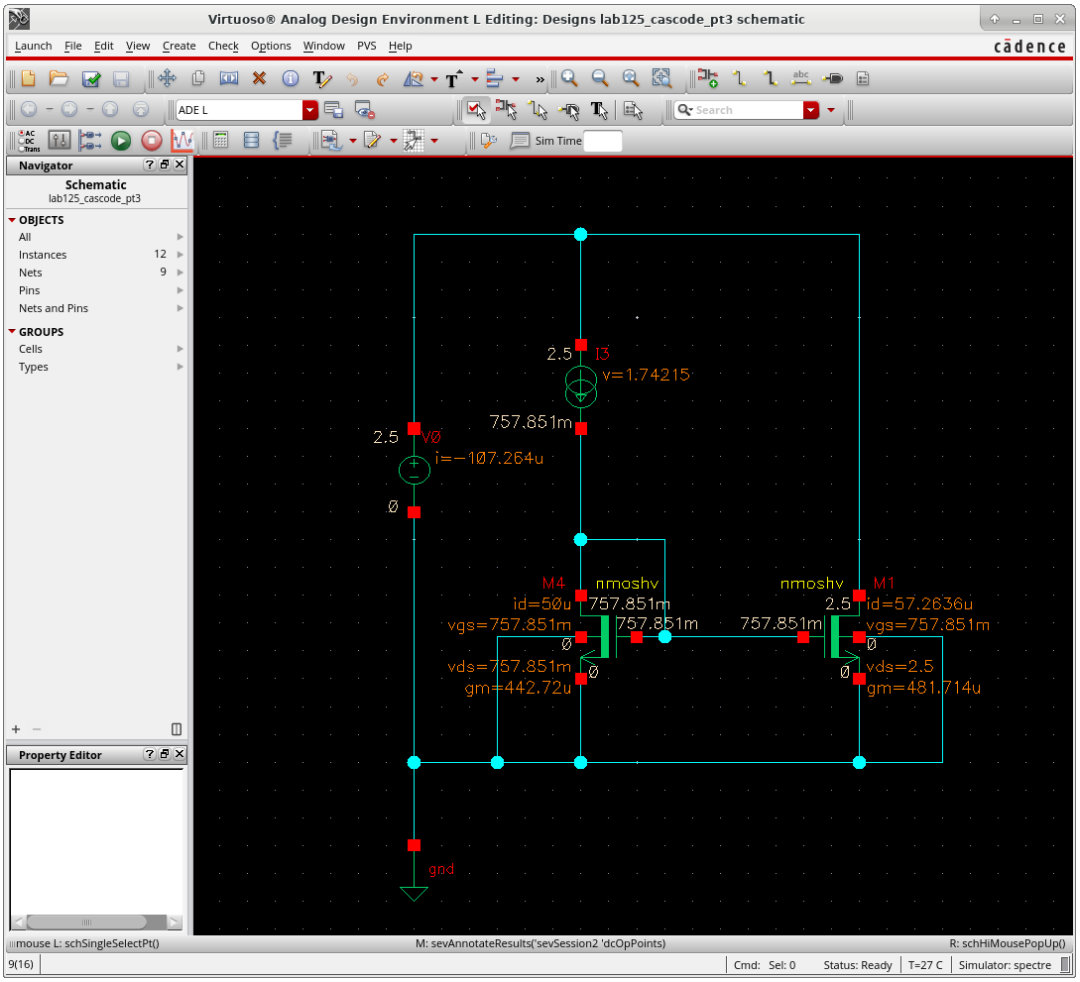
< Figure 147. The Choosing Analyses Form. Only DC Operating Point selected. >



< Figure 148. The ADE for a DC operating Point only Analysis. >

(6.2.5) From the ADE window choose **Menu: Results à Annotate à DC node voltages…** If you now look in the schematic window, you should see the node voltages annotated directly on the schematic. Again, from the ADE window, choose **Menu: Results à Annotate à DC operating points…** You should now see (along with the node voltages) additional DC operating

point information annotated on the schematic including the DC bias currents, NMOS device transconductance gm, IDS, VGS, and VDS. See Figure 5.



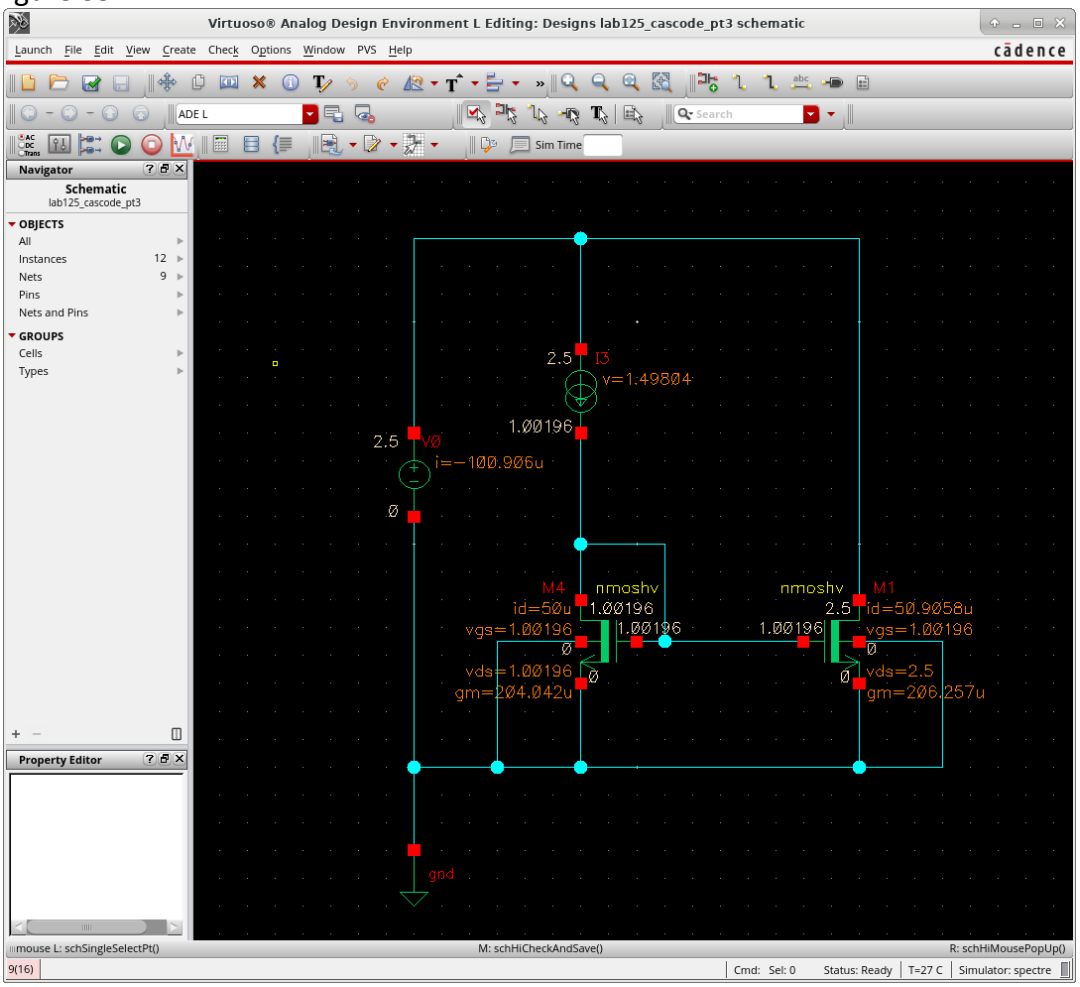
< Figure 149. DC Analysis – Save DC Operating Point. Annotated Schematic of the Basic Current Mirror. 10um/1um >

(6.2.6) **Discussion**. Note the annotated values for the drain (id) current for both devices. Despite both NMOS devices being identical, with identical V_{GS}, poor current mirroring (**50uA vs 57.3uA**) results from the vastly different values of vds (**0.758V vs 2.5V**) and the consequent effect of channel length modulation. As before, we can reduce the dependency of I_{DS} on V_{DS} by either increasing the length of the channel (making the effect of channel length modulation less sensitive) or by cascoding (shielding the mirror devices from any differences in V_{DS}).

6.3 Increasing the Channel Length to Improve Current Mirroring

(6.3.1) We will re-run the previous simulations but with the MOSFET channel length increased from 1um to 4um. Dimension both NMOS devices 10um/4um (**width** = *10u*, **length** = 4u). Leave all other attributes unchanged.

(6.3.2) From the Analog Design Environment window, re-run the DC Operating point analysis. See Figure 35.



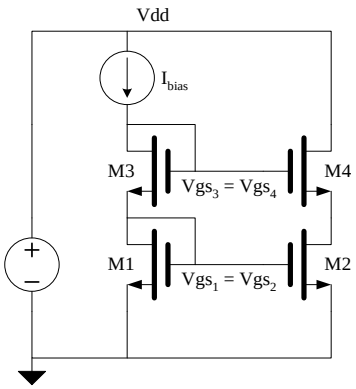
< Figure 150. DC Analysis – Save DC Operating Point. Annotated Schematic of the Basic Current Mirror. 10um/4um >

(6.3.3) **Discussion**. Increasing the channel length reduces the effect of channel length modulation despite the large difference in VDS (**1.002V vs 2.5V**). r_{ds} is increased (by reducing the dependency of IDS on VDS) such that the output device behaves more like an ideal current source. This improves current mirroring (**50uA vs 50.9uA**). However, since r_{ds} =Ve*L/ I_{ds} this strategy is highly current dependent. Where a current mirror is used as an active load for high-frequency circuits – a longer channel length may be an issue (especially where a proportional increase in device width is required to achieve a given gm).

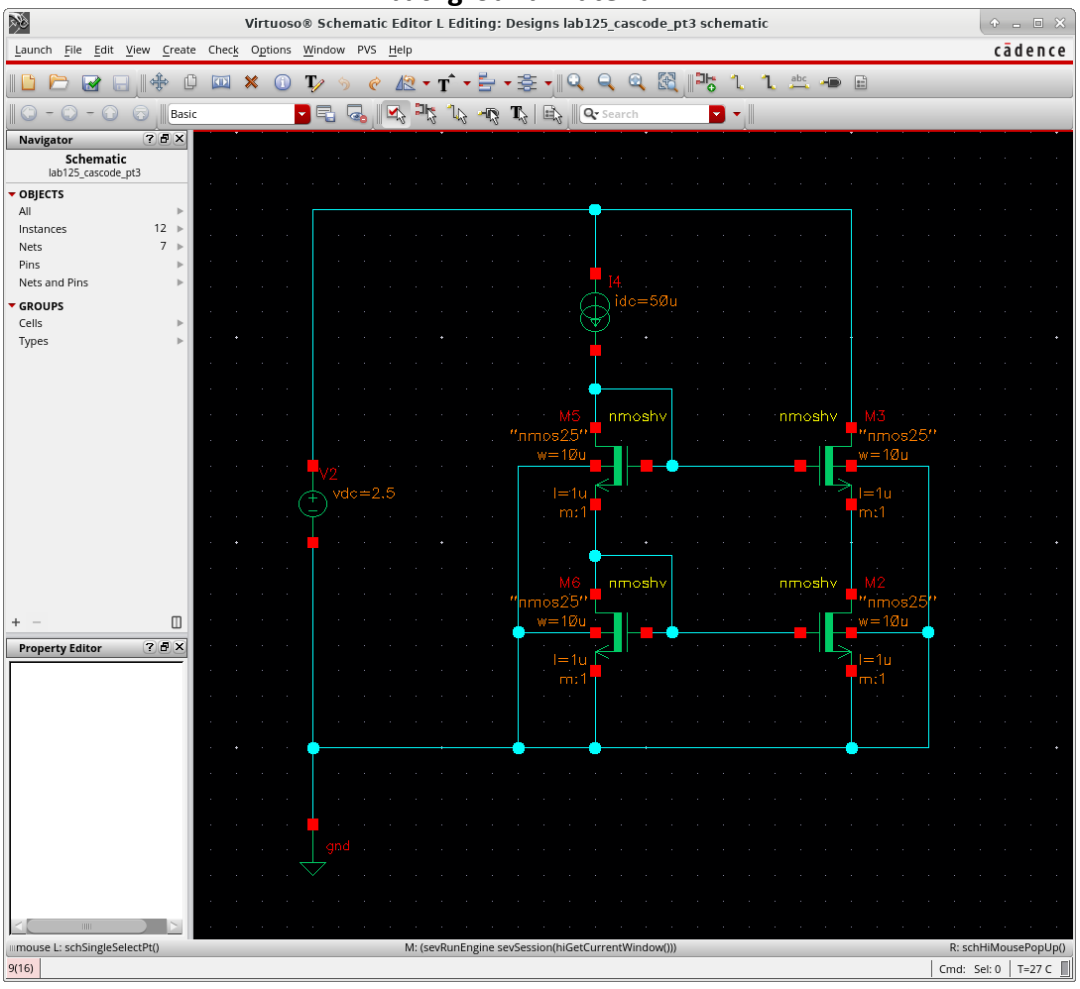
6.4 Cascoding the Basic Current Mirror to Improve Current Mirroring

(6.4.1) We will add a cascode stage to the basic current mirror in Figure 30 (31).

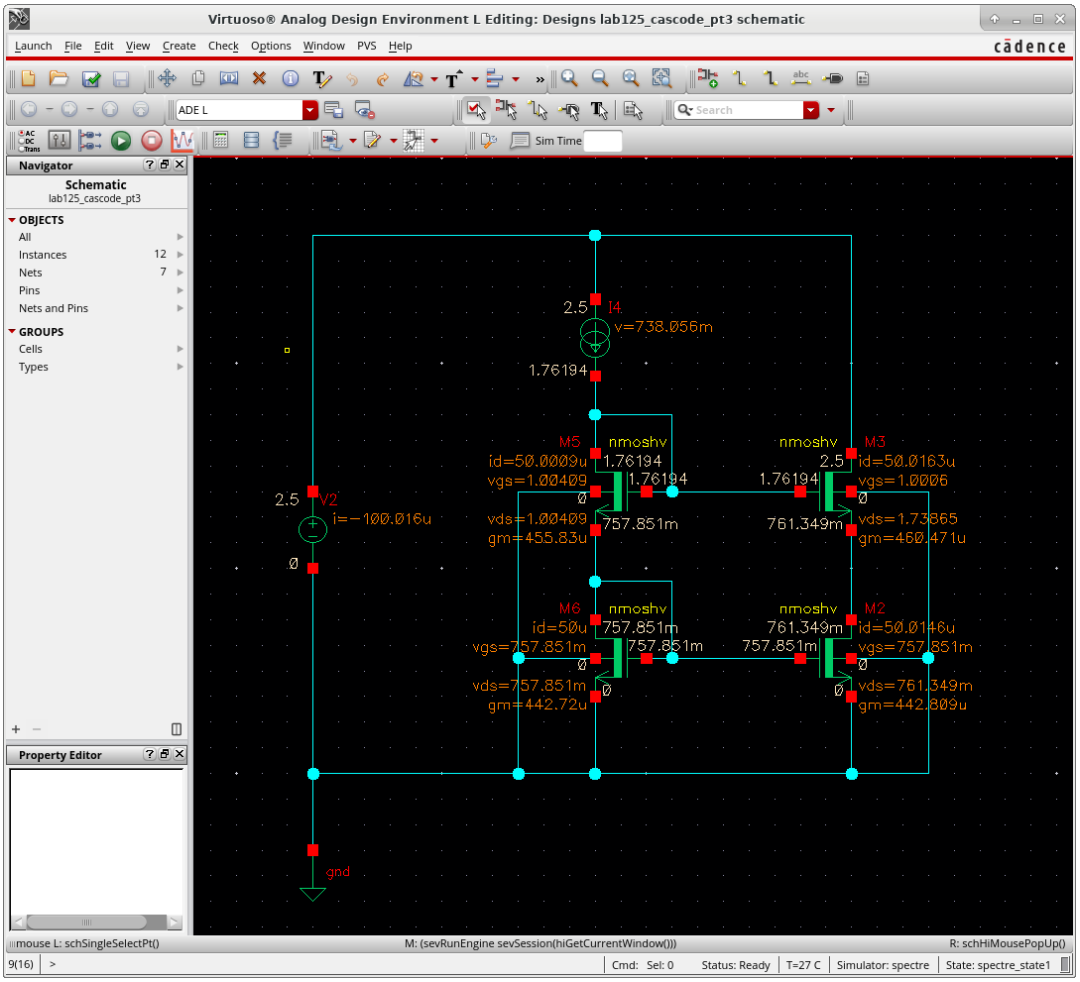
(6.4.2) Modify your existing testbench schematic by constructing a second mirror circuit alongside your existing circuit. The new circuit is the same as the previous one except for the addition of the identical cascode devices. See Figure 36.



< Figure 151. Cascode Current Mirror. Reproduced from Figure 19, section 4.3.2 of the background material. >



< Figure 152. Schematic of the Cascode Current Mirror. 10um/1um >



< Figure 153. DC Analysis – Save DC Operating Point. Annotated Schematic of the Cascode Current Mirror. 10um/1um >

(6.4.3) From the Analog Design Environment window, re-run the DC Operating point analysis. See Figure 38.

(6.4.4) **Discussion**. Note that the v_{ds} of the two lower devices are now much closer (**0.758V vs 0.761V**). This gives much better matching in mirror current (**50uA vs. 50.02uA**) for half the total gate area (with respect to the pair of 10u*4u devices) but at the expense of less output voltage headroom due to the saturation requirement of cascode devices. The designer will commonly want very accurate matching with currents and so will be willing to tolerate this. Note that other versions of the cascoded current mirror exist that offer greater voltage headroom for a similar mirror performance.

STUDENT TASK: Insert a separate, parameter-controlled, Vd for the output of the mirror and examine Iout/Vout. Then, by setting plot mode to 'append' plot the un-cascoded mirror and compare. Note that the mirrors are forced out of saturation near the same Vd but the cascode mirror has much better immunity to variations in Vd.

End of Lab!

Tutorial Lab 12.5 (Formatively Assessed): (…continued)

Symbol View Generation

7. Circuit Simulation Exercises: Part 4

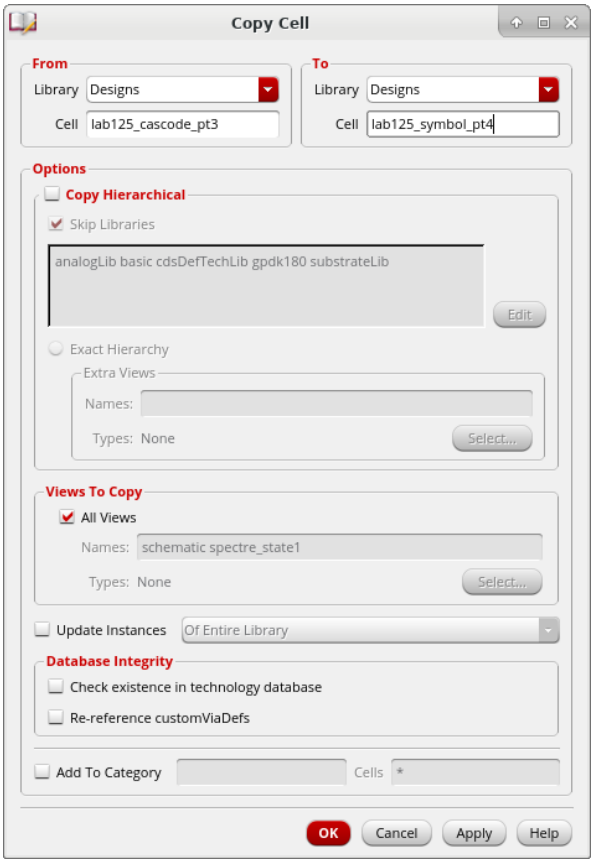
With guidance, you will

- Construct, instantiate and use in a simulation, a symbol for a current mirror circuit.

In electronic design, we will often want to abstract a complex schematic function or sub-function behind a symbol. This ensures only the interface to the function is made available to the rest of the design, with the complex (unnecessary) implementation details hidden away. This promotes the concept of the hierarchical design procedure (as opposed to a flat design), as a way of improving design reliability, manageability and reuse of components. In this part of the lab, we will look at the how we generate a symbol cellview for a given schematic.

7.1 Creating the Pinout of the Basic Current Mirror

(7.1.1) Make a copy of the basic current mirror schematic you created in the previous part of this lab (lab125_cascode_pt3). To create a copy from an existing schematic, go to the Library Manager window and select the cell you want to copy – in this case, lab125_cascode_pt3 from the Designs library. Right-click on the cell and select 'Copy…' from the context menu to invoke the 'Copy Cell' form. (You could also select **Menu: Edit à Copy** from the menu bar.)

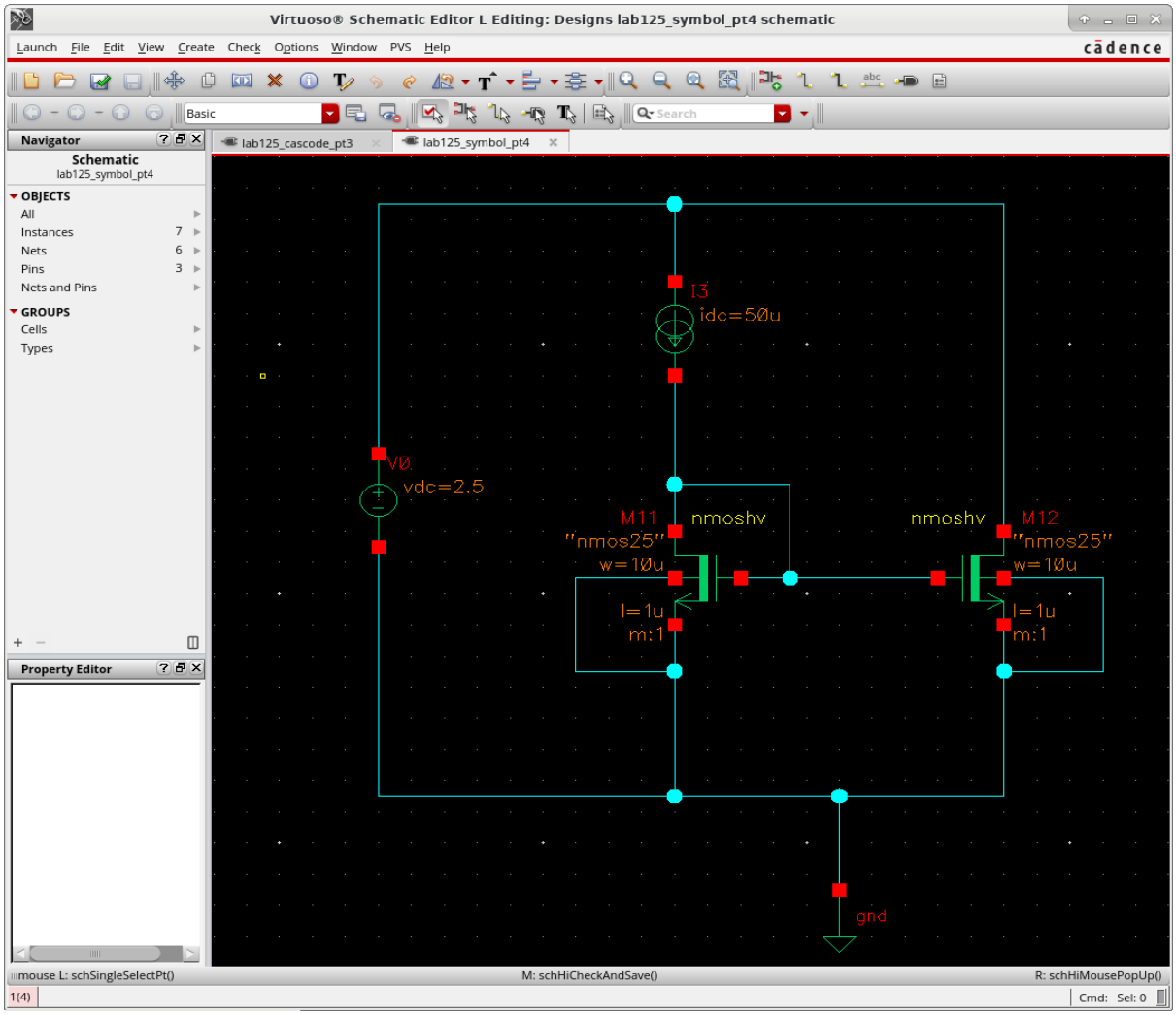


< Figure 154: The Copy Cell Form >

In the 'From' section of the copy cell form, select the source library and cell name and in the 'To' section choose the destination library and cell name (in this lab, use the Designs library in both the From/To sections). Give the new cell a meaningful name e.g., lab125_symbol_pt4. Leave the rest of the form unchanged and OK the form. In the library manager, in the Designs library, you should now see a new schematic cellview called 'lab125_symbol_pt4' (or whatever name you chose). See Figure 1.

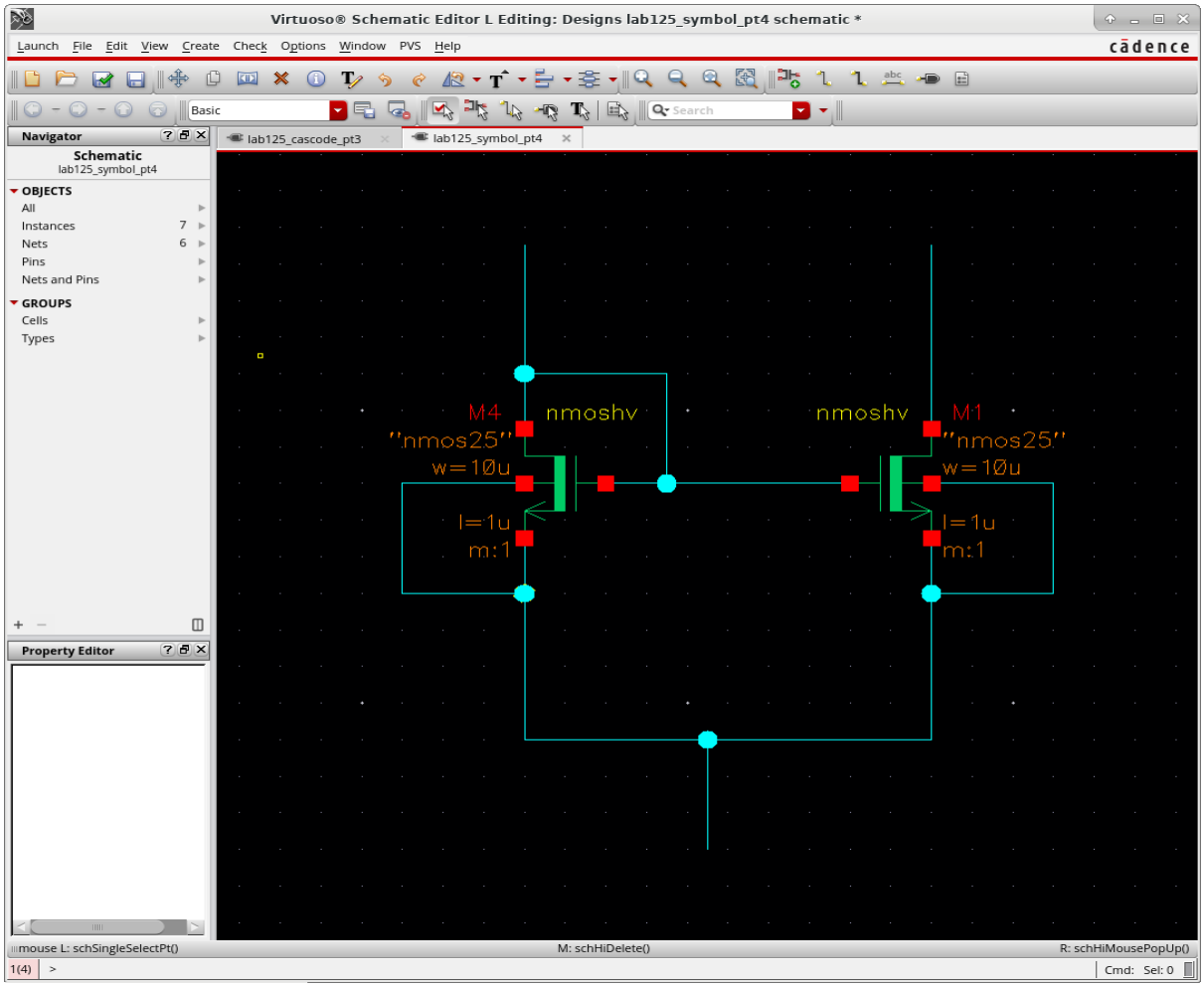
(7.1.2) **Note**: An alternative method of copying a schematic cell view, is to save a copy from the original schematic (lab125_cascode_pt3). In the Schematic Editor window from **Menu: File à Save a Copy…**, specifying the library, cell name, and cell type of the new cellview (e.g., lab125_symbol_pt4). Note, that in this case, once you have saved, a copy you remain working in your original schematic – you do not automatically switch to the new copy. You should see the new copy in the Library Manager – close the original (current) schematic and open the new schematic cellview.

(7.1.3) In your new schematic editor window, delete the cascode current mirror created previously. You should now only have the basic current mirror circuit in your schematic (lab125_symbol_pt4). See Figure 2.



< Figure 155. Schematic of the Basic Current Mirror. 10um/1um. >

(7.1.4) Delete the sundry/support components in the schematic – the 2.5V DC voltage supply (**vdc**), 50uA DC current reference (**idc**), and the ground reference (**gnd!**). See Figure 3.



< Figure 156. Schematic of the Basic Current Mirror with sundry/support components removed. >

(7.1.5) Add pins for the input, output, and ground (input/output) connections of the mirror circuit. From the schematic window, invoke the Create Pin form: use **Menu: Create à Pin...**, the '**Create Pin**' icon-button on the toolbar at the top of the schematic window, or the keyboard shortcut 'P'. You can also right-click anywhere in the empty space of the design canvas to invoke a context-menu and select the **Add Pin…** command from there.

When creating a pin you need to specify the name, direction, usage, and signal type of the pin. From the user manual,

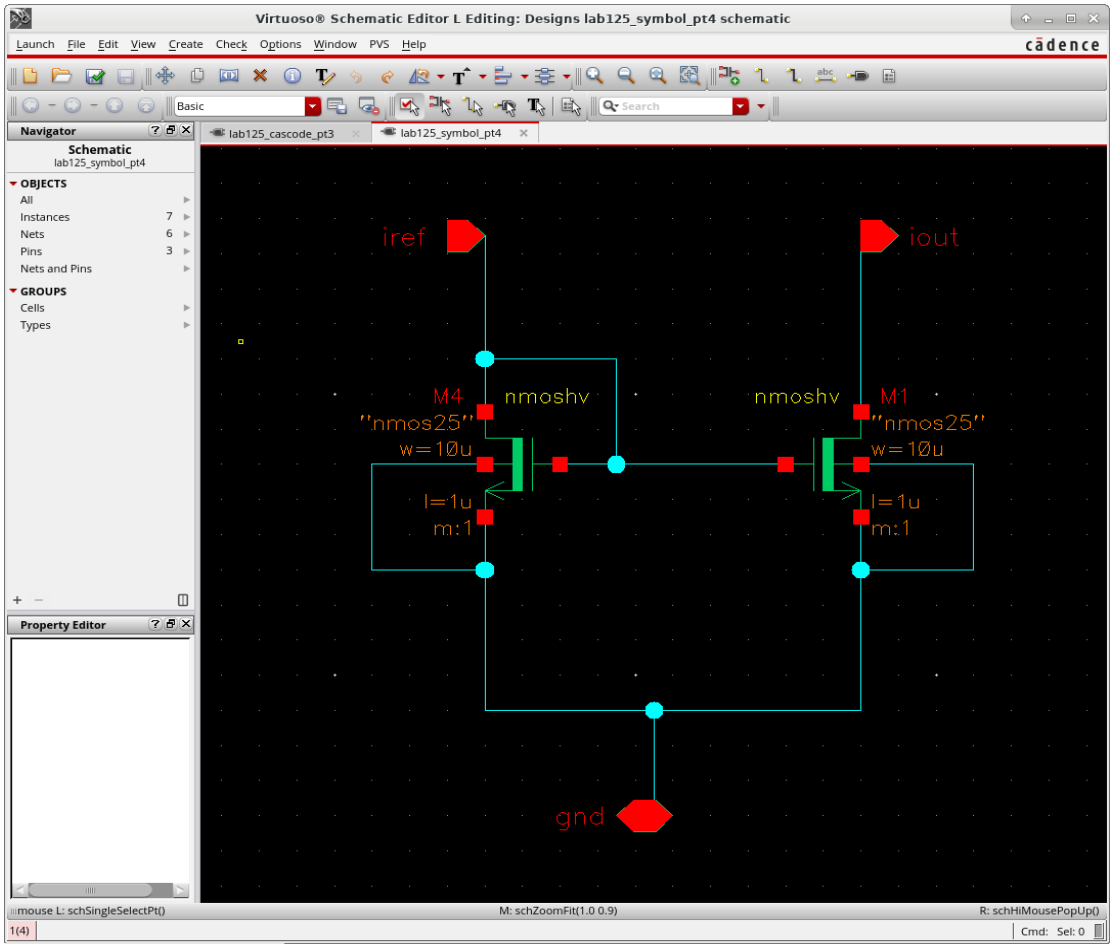
- **Names** specifies the names for the pins you want to add to the design and the order in which you want to place them. You can add more than one pin at a time as long as they are of all the same type/direction. To add more than one pin name, use a space to separate each name.

- **Direction** contains a drop-down list of the directions you can specify for the pins you want to add. For AICD, we will use only input pins, output pins, and inputOutput pins (for bidirectional connections such as for ground). Other options are available for specific applications.

- **Usage** contains a list of the pin types you can specify for the pins you want to add. The default is *schematic*. Use *offsheet* only when you are going to connect the added pins to another sheet in a multisheet schematic.

- **Signal Type** specifies the new pin's signal type. For this unit we will use 'signal' and 'ground' types only. Signal type setting is important in the areas of design verification, consistency checking for mixed-signal designs, and to also set routing constraints. Signal types to select from are *analog, clock*, *ground*, *power*, *reset*, *scan*, *signal* (default), *tieHi*, *tieLo*, *tieOff*, *optical*, *singleModeOptical*, or *multiMod*.

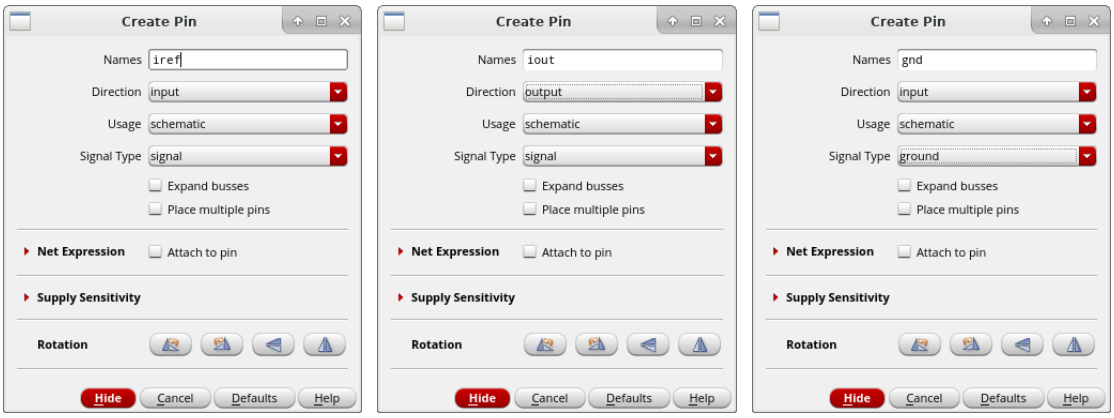
For further details of the Create Pin form, click the 'Help' button at bottom right corner of the form.



< Figure 157. Schematic of the Basic Current Mirror with IO and Ground Pins. >

(7.1.6) For the current reference branch of the mirror (the control branch), use an '*input*' pin, of '*signal*' type, and name it '***iref***'. For the output branch of the mirror (the controlled branch), use an '*output*' pin, of '*signal*' type, and name it '***iout***'. For the ground connection of the mirror, use an '*inputOuput*' pin, of '*ground*' type, and name it '***gnd***'. All pin usage types should be set to '*schematic*'. See Figures 4 and 5.

(7.1.7) Check and save you schematic using the **Check and Save** command. Remember to use the output area of the CIW to diagnose any problems with your schematic and/or simulations if there are any warning or error messages when checking, saving, or simulating your work.



< Figure 158. Create Pin Forms for Input, Output, and Ground >

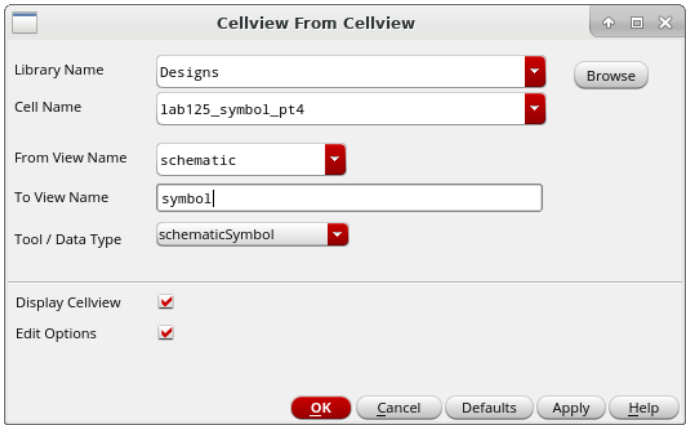
7.2 Generating the Symbol Cellview

(7.2.1) You should now have the completed schematic of the current mirror pinout as shown in Figure 4. It should check and save without any errors or warnings.

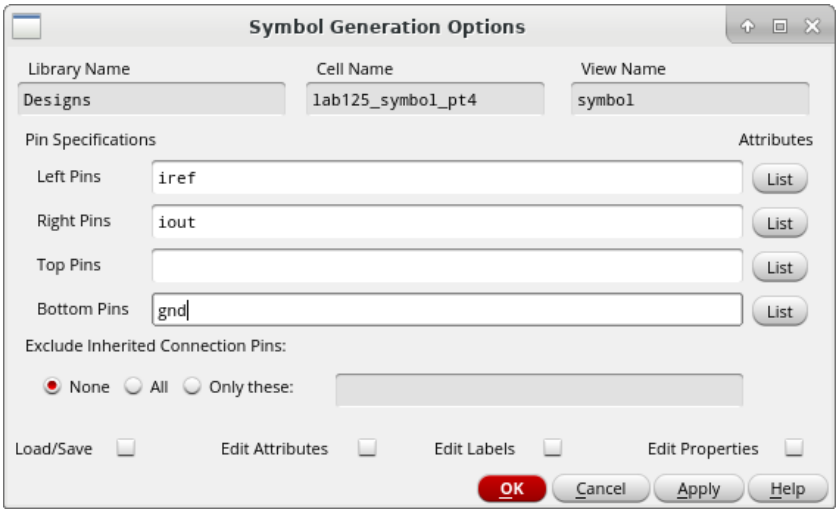
(7.2.2) From your schematic window (Figure 4), generate the symbol cellview using **Menu: Create à Cellview à From Cellview**… This will invoke the **Cellview From**. In this case, we will generate a symbol Cellview from a schematic cellview. The form may already be auto-filled in.

In the Cellview form, select the library and cell name of the schematic for which we want to generate a symbol. For the 'Library Name' select *Designs*, for the 'Cell Name' select *lab125_symbol_pt4*. Set the 'From View Name' to *schematic*, and the 'To View Name' to *symbol*. For the 'Tool / Data Type' select *schematic Symbol*. OK the form. See Figure 6.

When you OK the form, the **Symbol Generation Options form** will open. This is where you can set the initial properties and specifications of the symbol pinout. The pertinent aspect of this form are the pin positions on the symbol: left, right, top, and bottom. In schematic layout we generally arrange the pins on a component assuming a signal flow from left to right, and a current flow from top to bottom. In the associated text fields, put the *iref* pin (input) on the left, put the *iout* pin (output) on the right, and put the ground pin (inputOutput) on the bottom. Leave all other properties unchanged. OK the form. After a short while the symbol editor should open (there may be a slight delay). See Figure 7 and 8.



< Figure 159. Cellview From Cellview Form >



< Figure 160. Symbol Generation Options Form >

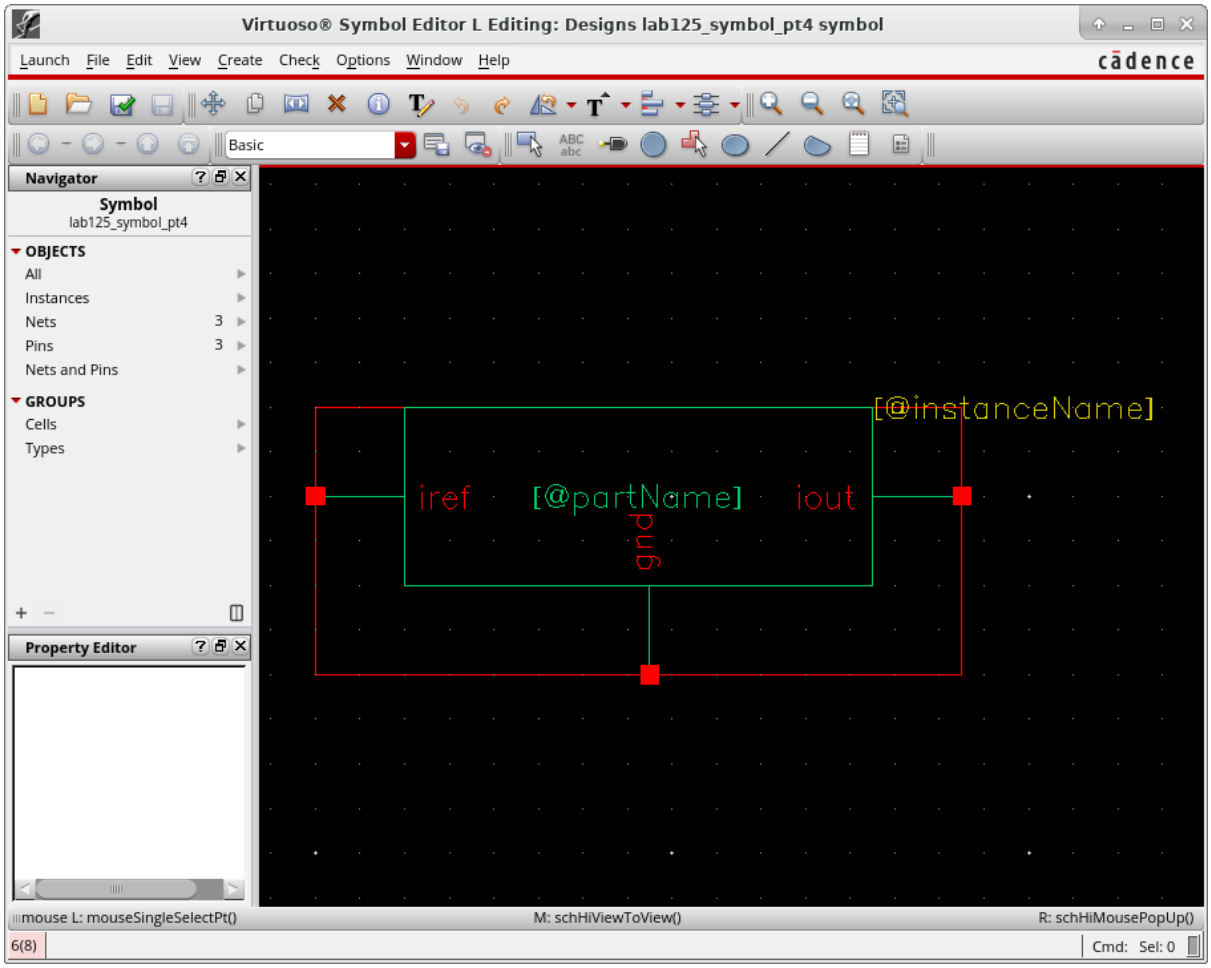
(7.2.3) Note: You can place multiple pins on a side using a name list separated by spaces.

(7.2.4) The symbol editor allows you to create and/or modify the symbol appearance. You can add or modify the basic shape of the symbol using lines/arcs, circles/ovals, rectangles, and polygons. Add or edit symbol pins. Add or edit symbol labels. Add or edit the symbol boundary/selection-box (the red outline box). Add or edit the part name and instance name display. Add/edit notes for documentation. And more.

One word of warning when adding or editing the symbol pins. The pin attributes were set-up in the schematic (using the create pin command). The symbol was then generated from the schematic. The number and properties of the pins (name, type, direction, and usage) of the symbol must match the number and properties of the pins defined in the schematic. If you change the pin definitions in the symbol view you must also update the schematic view to match – and vice versa. If the pin specifications do not match, a warning will be generated, and the simulation results may not be as expected.

The **@partName** and **@instanceName** are place holders for, respectively, the name of the cellview (in this case lab125_symbol_pt4), and the instance identifier that is automatically assigned to each device instantiated in a schematic (e.g., V1, R0, M0). Both of these @names will be set on the schematic at instantiation.

Finally, the red selection box around the symbol (Figure 8), sets the boundary of symbol when instantiated within a schematic. When you hover over or click on a symbol, the yellow/pink box that appears around the device is defined in the symbol editor by this red boundary / selection box. This also determines if a warning is generated if two symbols overlap (try overlapping to symbols and then check and save). You can delete the selection box altogether, but this is not recommended.



< Figure 161. Generated Symbol of the Basic Current Mirror. >

(7.2.5) You are free to modify the symbol as you wish – resizing, moving the names, adding shapes to the symbol – to observe the effect on the symbol layout. Remember to **check and save** your symbol (when it is first generated, and every time you make a modification).

7.3 Simulating the Symbol Cellview Using a Testbench

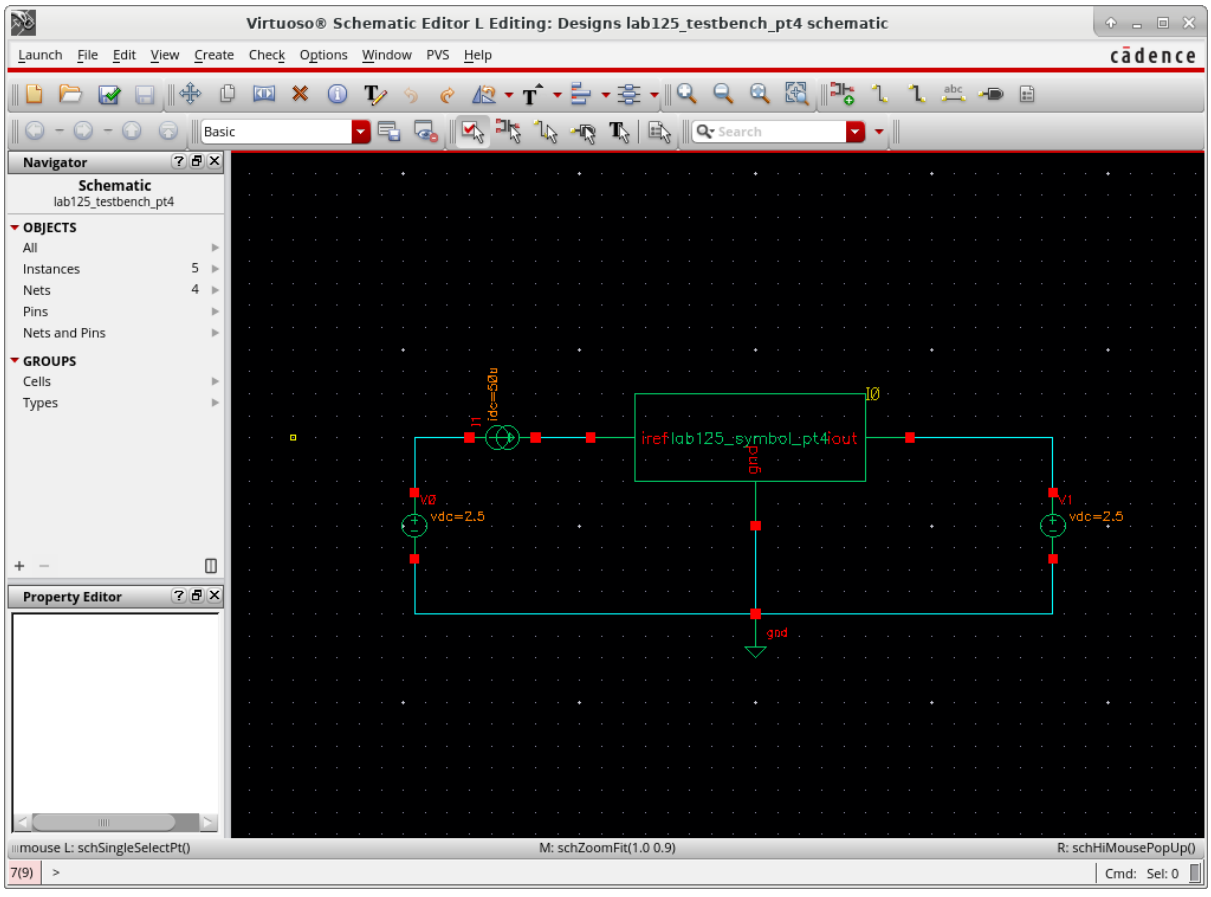
(7.3.1) In order to demonstrate the behaviour of the basic current mirror symbol view, we will create a testbench.

(7.3.2) Create a new schematic. Use the New File form as you have done previously – ensuring you are in your Designs library – giving your new schematic a meaningful name such a 'lab125_testbench_pt4'.

(7.3.3)) Using the Add Instance form, in your testbench schematic editor window, create the circuit given in Figure 9. Instantiate two ideal DC voltage sources (**vdc**) and one ideal DC current source (**idc**) from the analogLib library. Also, instantiate one instance of the symbol you have just created – this will be the symbol view of part *lab125_symbol_pt4* in your *Designs* library.

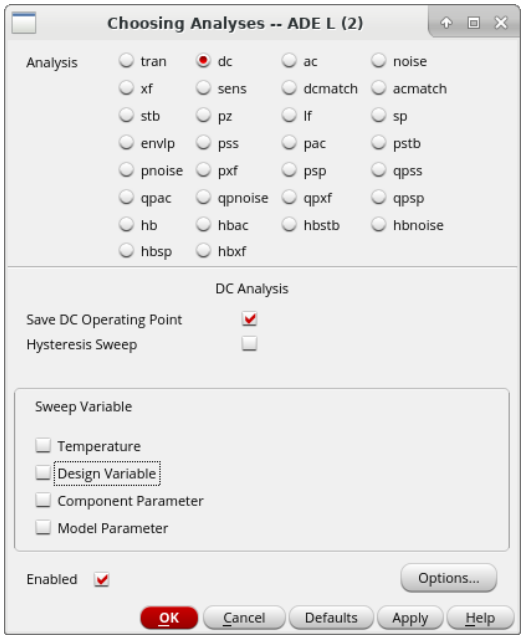
Set the DC supply voltages to 2.5V (**DC Voltage** = 2.5) and set the DC current reference to 50uA (**DC current** = 50u).

Note: In Figure 9, we are using the second 2.5V **vdc** as both the supply to the output branch of the mirror and as a current sink. The latter not strictly necessary, but it simplifies the layout of the testbench.

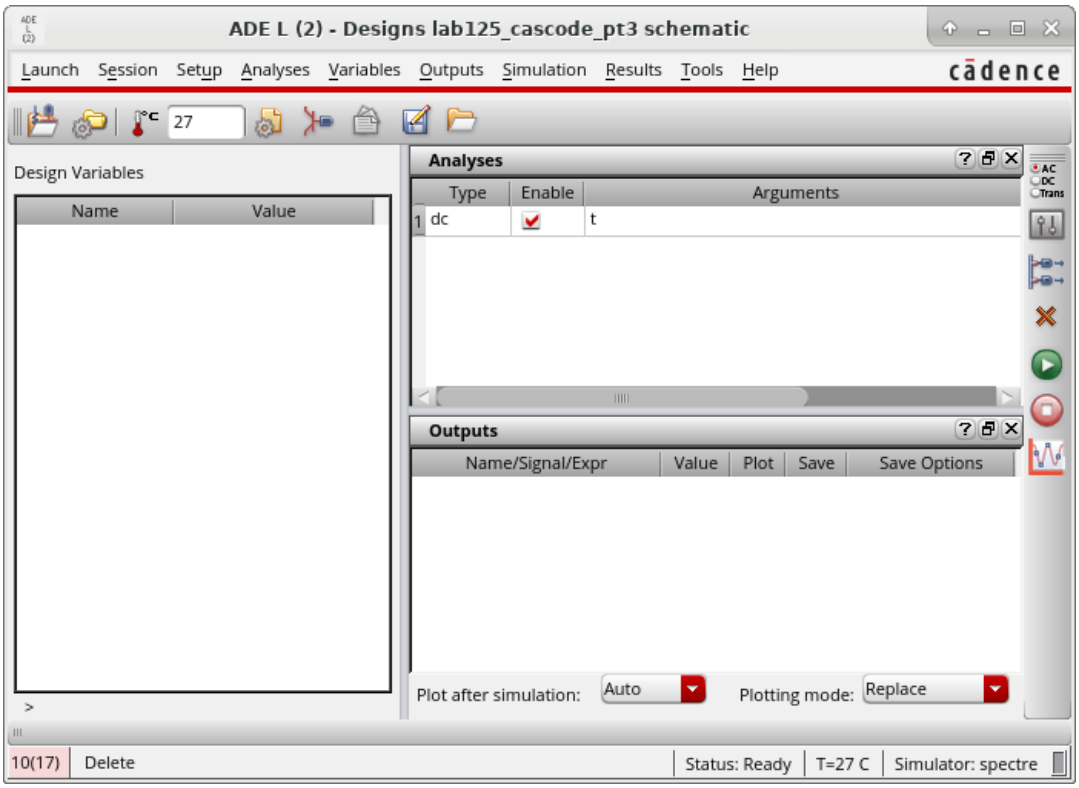


< Figure 162. Testbench for the Basic Current Mirror Symbol. >

(7.3.4) We will simulate the current mirror using a DC Analysis. From the Schematic Editor window, launch the Analogue Design Environment (ADE) from **Menu: Launch à ADE L**.



< Figure 163. The Choosing Analyses Form. Only DC Operating Point selected. >



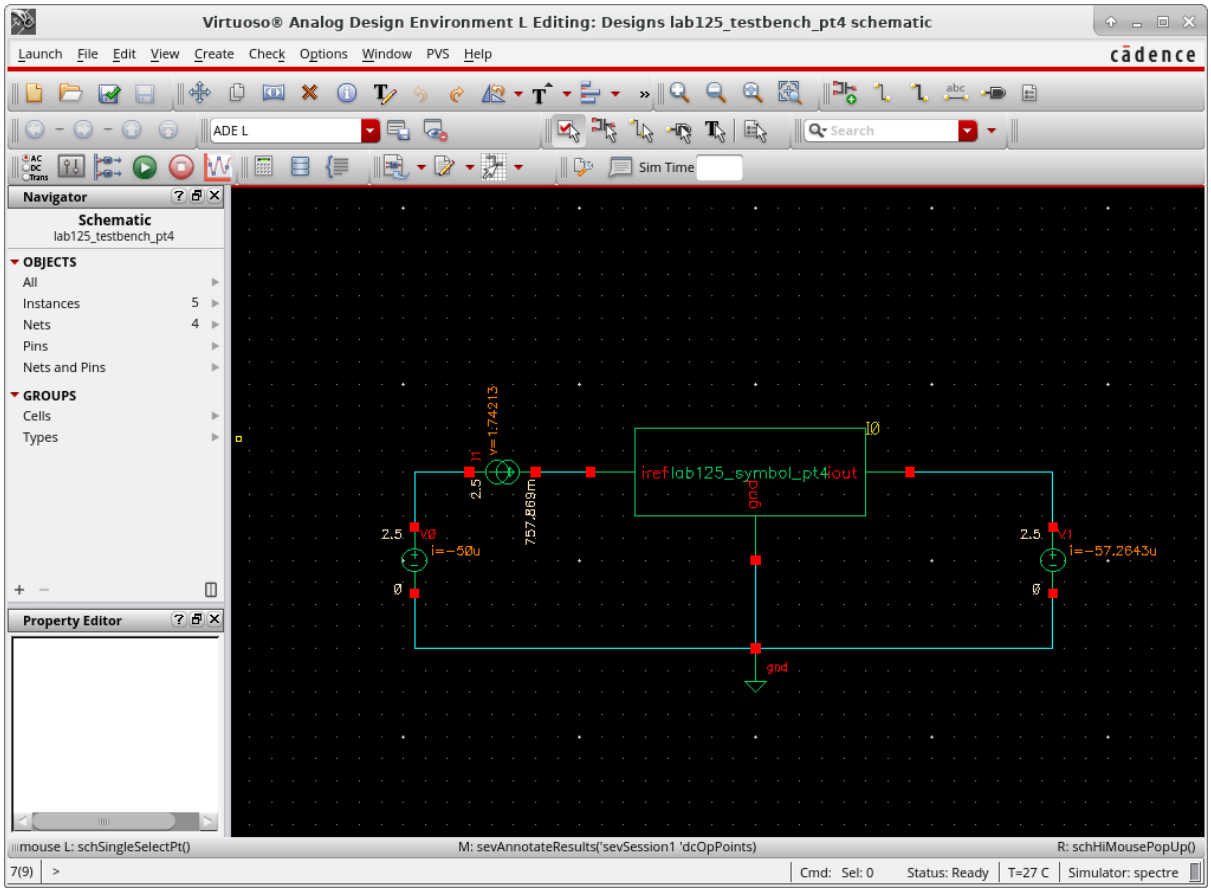
< Figure 164. The ADE for a DC operating Point only Analysis. >

(7.3.5) From the ADE window, go to **Menu: Analyses à Choose…** or use the **Choose Analyses** button on the right-side of the ADE window (top button). The 'Choosing Analyses – ADE L' form will open. On the form, select the '**dc**' option button. Check the 'Save DC Operating Point' box. OK the form.

(7.3.6) We are only simulating for the DC operating point, so we do not need to select any outputs to be plotted.

(7.3.7) In the Analog Design Environment window, run the '**Netlist and Run**' command by clicking on the green play button. The simulation should run without error. The text output window should appear but there will be no waveform window.

(7.3.8) From the ADE window choose **Menu: Results à Annotate à DC node voltages…** If you now look in the schematic window, you should see the node voltages annotated directly on the schematic. Again, from the ADE window, choose **Menu: Results à Annotate à DC operating points…** You should now see (along with the node voltages) additional DC operating point information annotated on the schematic including the DC bias currents.



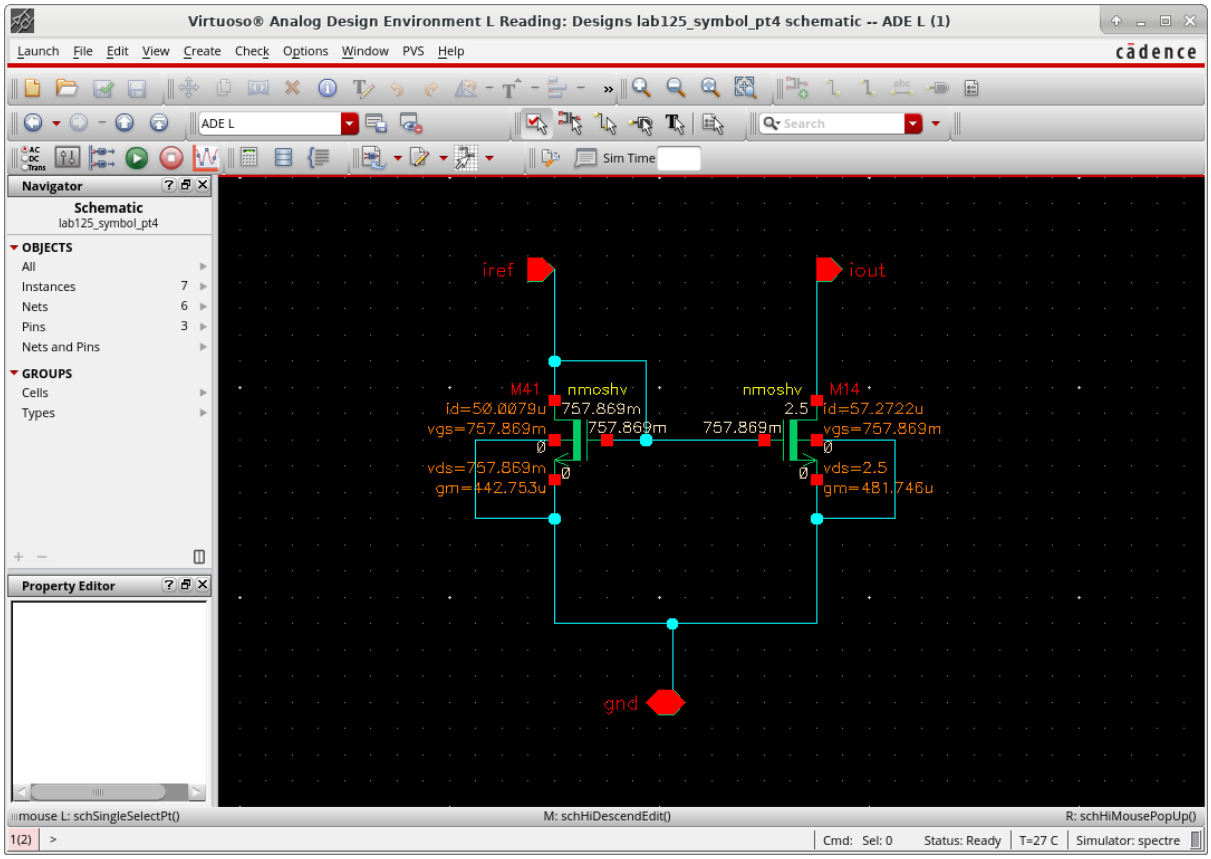
< Figure 165. DC Analysis – Save DC Operating Point. Annotated Testbench for the Basic Current Mirror Symbol. >

(7.3.9) You can look beneath the symbol abstraction by right-clicking on the symbol and choosing **Descend Edit** (keyboard shortcut shift+E) or **Descend Read** (keyboard shortcut E). You can also select the symbol and use **Menu: Edit → Hierarchy → Descend Edit**… or **Menu: Edit → Hierarchy → Descend Read**… This will invoke the Descend form, where you can choose to open the underlying schematic in the current window tab or in a new window tab.



< Figure 166. The Descend (into sub-level) Form. >

Note that Descend edit allows you to edit the schematic beneath the symbol when you descend into it (this can be convenient when you want to make quick changes to a symbol and have it applied to every instance in the schematic). Descend read does not allow you to modify the schematic when you descend into it (this is useful if you don't want to change the schematic or if a schematic is protected i.e. read only).



< Figure 167. Descended View of the Basic Current Mirror Symbol. >

If you descended into the symbol within the current window tab, you can return to the schematic one level above by right-click on the blank canvas and selecting the **Return** command (keyboard shortcut ctrl+E). You can also use **Menu: Edit → Hierarchy → Return**… If you have descended several levels, you can return straight to the top-level module using **Menu: Edit → Hierarchy → Return To Top…**

(7.3.10) **Discussion**. Note that the DC voltages and currents are (almost) the same as in the original simulation (Figure 34) – VGS = 0.757869V (0.757851V) and IDS = 57.2643uA (57.2636uA). The slight difference in the final two digits occurs since we are using two separate **vdc**'s for the supply in testbench, as opposed to one **vdc** in the original simulation.

(7.3.11) Finally, now that you have a hierarchical (multi-level) schematic, you have to make sure when checking and saving that all levels are properly checked/saved. If a lower level (within a symbol) isn't checked and saved, even if the top-level module is, you will get an error when trying to simulate.

Warning: making a change in a subcircuit schematic results in a change in every instantiation of its symbol.

End of Lab!